Tugas Besar IF4031

Arsitektur Aplikasi Terdistribusi

ARLOW:

Agregasi Real-time Layanan Operasi Waspada

Laporan Milestone 1

Kelompok **nasigoreng**

Muhammad Aulia Azka	13523137
Arlow Emmanuel Hergara	13523161
Fachriza Ahmad Setiyono	13523162

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

10 Oktober 2026

Kontribusi Anggota

NIM	Nama	Kontribusi
13523137	Muhammad Aulia Azka	*Diisi anggota: perancangan, implementasi, dan penulisan laporan.*
13523161	Arlow Emmanuel Hergara	*Diisi anggota: perancangan, implementasi, dan penulisan laporan.*
13523162	Fachriza Ahmad Setiyono	*Diisi anggota: perancangan, implementasi, dan penulisan laporan.*

Kontribusi diisi manual oleh anggota agar pembagian perancangan, implementasi, dan penulisan

mencerminkan pekerjaan yang benar-benar dilakukan.

Daftar Isi

Kontribusi Anggota **i**

1 Gambaran Sistem **1**
1.1 Tujuan dan Ruang Lingkup Milestone 1 1
1.2 Arsitektur dan Batas Layanan 1
1.3 Teknologi dan Alasan Pemilihan 3
1.4 Asumsi dan Konfigurasi 4
1.5 Ringkasan Menjalankan Sistem 5
1.6 Strategi Verifikasi 5

2 P1: Interoperabilitas Data **6**
2.1 Rancangan Solusi 6
2.2 Justifikasi dan Alternatif 7
2.3 Batasan Solusi 8
2.4 Proof-of-Concept dan Keterlacakan 8

3 P2: Konkurensi dan Ketersediaan **13**
3.1 Rancangan Solusi 13
3.2 Justifikasi dan Alternatif 14
3.3 Batasan Solusi 14
3.4 Proof-of-Concept dan Keterlacakan 15

4 P3: Autentikasi dan Trust **21**
4.1 Rancangan Solusi 21
4.2 Justifikasi dan Alternatif 23
4.3 Batasan Solusi 23
4.4 Proof-of-Concept dan Keterlacakan 24

5 P4: Layanan dan Penyimpanan **30**
5.1 Rancangan Solusi 30
5.2 Justifikasi dan Alternatif 31
5.3 Batasan Solusi 32
5.4 Proof-of-Concept dan Keterlacakan 32

6 P5: Distribusi Event **38**
6.1 Rancangan Solusi 38
6.2 Justifikasi dan Alternatif 39
6.3 Batasan Solusi 40
6.4 Proof-of-Concept dan Keterlacakan 41

Daftar Pustaka **45**

Bab 1

Gambaran Sistem

1.1 Tujuan dan Ruang Lingkup Milestone 1

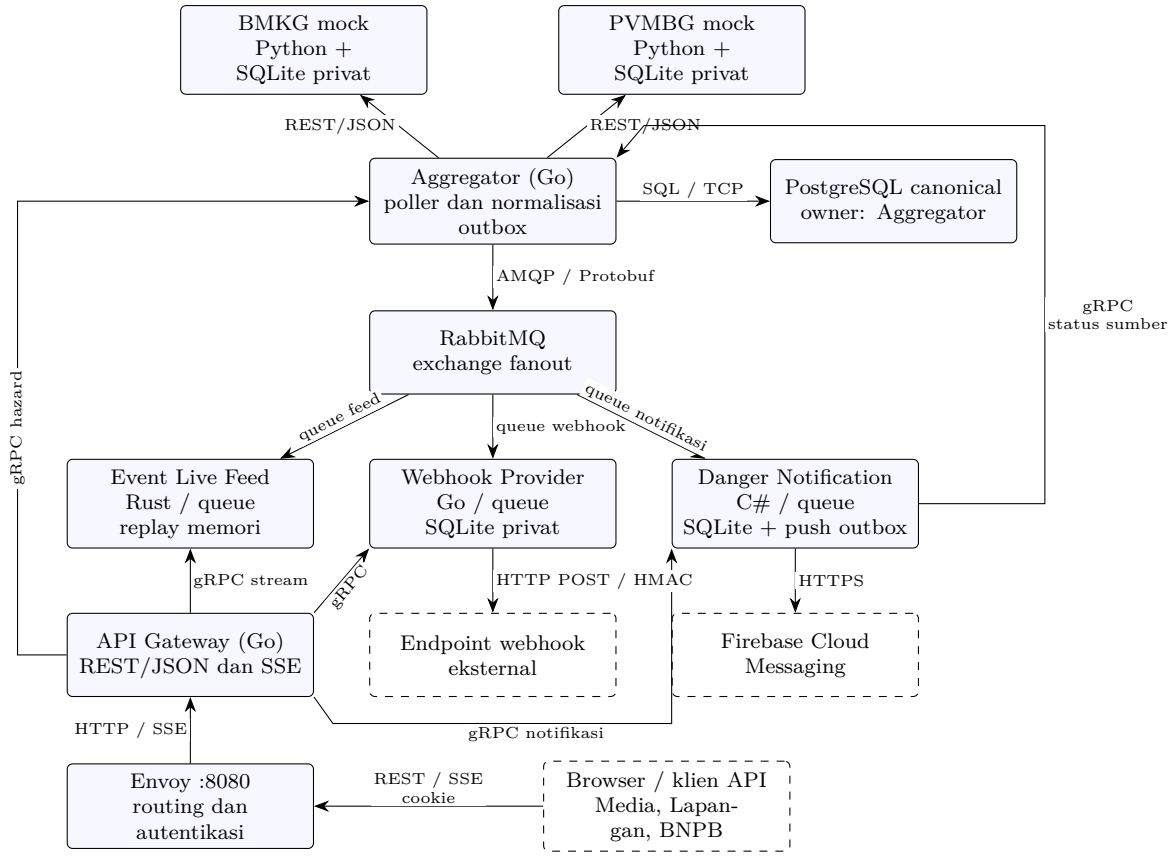
ARLOW: Agregasi Real-time Layanan Operasi Waspada mengintegrasikan kejadian gempa dan aktivitas gunung api dari mock BMKG dan PVMBG menjadi `HazardEvent` kanonik. BNPB mengambil data dengan polling, menyimpan hasil normalisasi, menyediakan API bagi klien, dan mendistribusikan publikasi yang sama kepada layanan live feed, webhook, serta notifikasi bahaya. Pengguna terdiri atas Media, Tim Lapangan, dan BNPB Pusat; hak akses ditentukan server berdasarkan identitas masing-masing.

Milestone ini berfokus pada interoperabilitas data (P1), isolasi kelambatan dan kegagalan sumber (P2), autentikasi dan otorisasi (P3), pemisahan layanan beserta penyimpanannya (P4), dan distribusi event melalui pub/sub (P5), sesuai spesifikasi tugas [1]. Data bencana adalah simulasi untuk pengujian, bukan informasi operasional kebencanaan. Klien membaca data hazard; operasi tulis terbatas pada pengaturan langganan, sesi, dan kontrol mock untuk demo.

Laporan diperbarui pada 10 Oktober 2026 berdasarkan snapshot repository `4a1973b`, termasuk bukti manual P1–P5, dua keluaran k6, laporan suite otomatis, serta empat screenshot tambahan P3/P4. Bukti baru memperlihatkan penolakan kredensial silang, refresh sesi OIDC pada browser, dan build Notification dengan `--no-deps`. Bukti historis tetap dicantumkan dengan tanggal aslinya. Penjelasan implementasi ditelusuri ke kode dan Compose pada snapshot tersebut; pembaruan dokumen tidak menjalankan ulang pengujian. Cakupan bukti dijelaskan pada bab terkait; inventaris berkas tersedia di `docs/tex/evidence/manifest.json`.

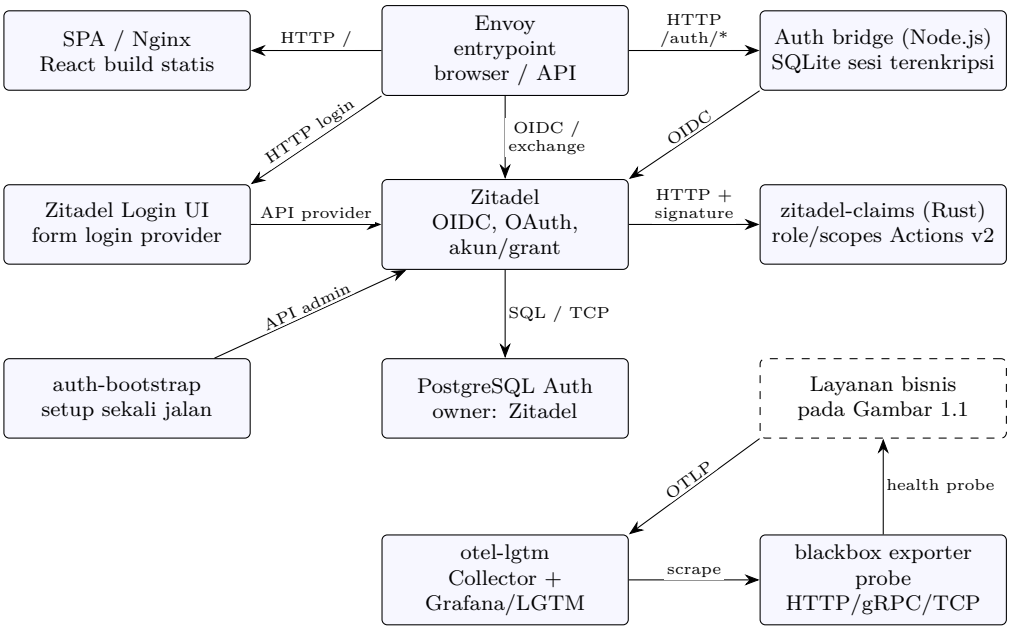
1.2 Arsitektur dan Batas Layanan

Gambar 1.1 menunjukkan jalur data, sedangkan Gambar 1.2 melengkapi komponen identitas dan observabilitas. Setiap kotak bergaris utuh merupakan container terpisah, kecuali browser dan tujuan eksternal yang ditandai khusus. SQLite dan buffer memori yang ditulis di dalam kotak dimiliki layanan tersebut, bukan database bersama. Kotak PostgreSQL adalah container database terpisah dengan pemilik logis tunggal.



Gambar 1.1: Container dan aliran data. Panah REST/gRPC mengikuti pemanggil ke penerima; panah AMQP mengikuti aliran publikasi.

1. Aggregator melakukan polling REST/JSON ke kedua mock dengan kredensial berbeda. Masing-masing mock membangkitkan data sendiri dan menyimpannya dalam SQLite privat. Tidak ada push langsung dari mock ke BNPB.
2. Aggregator memetakan data menjadi format kanonik. Hazard dan outbox dimasukkan melalui transaksi PostgreSQL; publisher mengirim envelope Protobuf dari outbox ke RabbitMQ.
3. RabbitMQ membagikan publikasi ke queue tiap consumer. Feed menyediakan replay dan stream, Webhook Provider mengirim HTTP callback, dan Danger Notification menyimpan notifikasi serta push outbox pada SQLite privat sebelum worker mengirim melalui Firebase Cloud Messaging (FCM).
4. Browser mengakses satu origin melalui Envoy. API Gateway Go menerjemahkan REST/JSON dan SSE menjadi pemanggilan gRPC ke layanan bisnis. Pembacaan hazard mengambil canonical store melalui Aggregator, bukan memanggil mock secara sinkron.



Gambar 1.2: Container identitas, penyajian frontend, bootstrap, dan observabilitas. Komunikasi ke Zitadel mencakup OIDC, introspeksi, serta token exchange.

Kelompok komponen *Auth* pada rancangan awal direalisasikan oleh Zitadel, hosted Login UI, Auth bridge, dan target Actions `zitadel-claims`. Auth bridge hanya menangani `/auth/*`; permintaan API bisnis melewati Envoy dan API Gateway. Bootstrap adalah container sekali jalan untuk menyiapkan proyek, akun, grant, dan client OIDC. Konfigurasi observabilitas menggunakan container `otel-lgtm` serta `blackbox`, bukan container Prometheus dan Jaeger terpisah seperti sketsa awal. PostgreSQL Auth berbeda dari PostgreSQL canonical store. Kotak putus-putus layanan bisnis pada diagram kedua merujuk kelompok container pada diagram pertama, bukan container tambahan.

1.3 Teknologi dan Alasan Pemilihan

Komponen	**Teknologi**	**Alasan dan tanggung jawab**
BMKG dan PVMBG	Python 3.12, HTTP standard library, SQLite	Dependensi ringan; thread request memungkinkan health/admin tetap merespons saat request data menunggu. File database mempertahankan seed dan event setelah restart.
Frontend	React, TypeScript, Vite, Nginx	SPA dengan tampilan login, filter/daftar/detail hazard, status sumber, live feed, notifikasi, dan webhook; Nginx menyajikan hasil build statis.
Edge dan REST API	Envoy; Go, chi, oapi-codegen	Edge melakukan routing dan pertukaran token. Go menjaga bentuk JSON, status HTTP, validasi OpenAPI, dan framing SSE yang berbeda dari ProtoJSON.
Aggregator	Go, gRPC, PostgreSQL 17	Goroutine untuk polling independen; transaksi dan indeks relational untuk data inti, JSONB untuk atribut dinamis.

Komponen	Teknologi	Alasan dan tanggung jawab
Event Live Feed	Rust, Tokio, Tonic	Stream asinkron dan buffer replay berbatas; API Gateway mengubahnya menjadi SSE untuk browser.
Webhook Provider	Go, SQLite WAL	Menyimpan subscription, hasil delivery, dan deduplikasi; worker mengirim HTTP callback bertanda tangan.
Danger Notification	C#, ASP.NET gRPC, EF Core SQLite, Firebase Admin	Menyimpan notifikasi dan push outbox privat, menyediakan proyeksi summary/full, serta mengirim push FCM melalui worker. Status kesegaran sumber dibaca melalui Aggregator.
Identitas	Zitadel, PostgreSQL, Node.js 24, Rust	Provider mengelola akun; bridge menjalankan OIDC dan sesi SQLite terenkripsi; Rust Actions memasok role/scope.
Distribusi event	RabbitMQ 4, AMQP, Protobuf	Fanout dan queue independen memisahkan produsen dari consumer serta menyediakan acknowledgement.
Observabilitas	OpenTelemetry, otel-lgtm, blackbox exporter	Pengiriman trace/metric/log dan probe HTTP/gRPC/TCP; dashboard serta alert diprovisikan melalui berkas repo.
Orkestrasi	Docker Compose	Batas proses, jaringan, volume, dan dependensi dinyatakan di satu stack utama; Compose khusus dipakai untuk pengembangan parsial.

1.4 Asumsi dan Konfigurasi

1. Instalasi M1 berjalan pada satu host Docker. Port publik default adalah BNPB 8080, BMKG 8081, PVMBG 8082, Grafana 3000, serta RabbitMQ Management 15672. Port mock dalam container adalah 8080.
2. Host `localhost` dan `auth.localhost` menunjuk mesin demo. HTTP diizinkan secara eksplisit untuk lingkungan lokal; ini bukan konfigurasi TLS produksi.
3. Setiap mock menyiapkan minimal 20 record pada database baru. Interval event default 10 detik; polling contoh 3 detik. Semua nilai latensi konfigurasi merupakan parameter simulasi, bukan hasil benchmark.
4. ID sumber dan timestamp menjadi dasar watermark polling inklusif. Tabel referensi gunung statis merupakan data milik BNPB. Penerimaan field tambahan tidak menjamin kompatibilitas perubahan tipe atau penggantian nama field wajib.
5. Aggregator adalah pemilik logis canonical store untuk operasi bisnis. Layanan lain memakai gRPC atau event; Notification membaca notifikasi dari SQLite miliknya. Collector juga mengakses PostgreSQL untuk metrik. Distribusi environment Compose belum membatasi kredensial database secara eksklusif pada Aggregator; batas isolasi ini dijelaskan pada P4.
6. Akun pengguna di-seed pada Zitadel, bukan SQLite mock ataupun frontend. Tiga akun manusia memiliki role terpisah; service user untuk tes end-to-end memiliki kredensial tersendiri.
7. Notifikasi push memerlukan proyek Firebase dan service-account JSON lokal; konfigurasi web publik diperlukan untuk push browser. Tanpa dependensi tersebut, demo push belum dapat dianggap terbukti.
8. Restart mempertahankan volume bernama. Menghapus volume menghilangkan data persisten. Replay feed berada di memori, dan RabbitMQ belum memiliki mount data persisten

eksplisit pada Compose saat ini.

1.5 Ringkasan Menjalankan Sistem

Prasyarat dasar adalah Docker Desktop/Engine yang aktif dan Python 3.12+ untuk helper setup. Siapkan konfigurasi lokal dari contoh; helper mempertahankan secret yang sudah tersedia. Helper mock hanya mengisi dua kredensial sumber, sehingga nilai `REPLACE_*` lainnya dalam `.env` harus diprovisikan sesuai pemiliknya. Isi kredensial broker dan token internal yang digunakan; sesuaikan DSN database dengan definisi Compose. Sediakan file Firebase pada `FIREBASE_CREDENTIALS_PATH`. Nilai privat tidak dicantumkan dalam laporan.

```
python tools/setup_mock_env.py
python tools/setup_auth_env.py
docker compose --env-file .env --env-file .env.auth -f compose.yaml config --quiet
docker compose --env-file .env --env-file .env.auth -f compose.yaml up --build -d --wait
```

Perintah terakhir merupakan satu perintah orkestrasi setelah prasyarat konfigurasi dipenuhi. Buka `http://localhost:8080`, pilih masuk melalui Zitadel, lalu gunakan akun `media-demo`, `lapangan-demo`, atau `bnpb-demo`. Password berasal dari `.env.auth`. Dashboard Compose utama menggunakan `VITE_DATA_MODE=gateway`; fixture lokal hanya untuk mode pratinjau/stack Auth terpisah. Login UI milik Zitadel tidak mengubah frontend BNPB menjadi SSR.

Panduan operasional tersedia di `README.md`, `docs/mock-testing.md`, `docs/auth-testing.md`, `docs/e2e-testing.md`, dan `docs/load-testing.md`. Untuk layanan BNPB yang sudah berkembang, gunakan Compose dan README layanan terbaru bila uraian status pada README root masih tertinggal. Stack parsial `compose.mocks.yaml`, `compose.auth.yaml`, serta `compose.frontend.yaml` tetap berguna untuk pengembangan, tetapi bukan pengganti pengujian integrasi stack utama.

1.6 Strategi Verifikasi

Penanda E1, E2, dan E3 merujuk hasil historis tes mock, browser Auth, dan unit Aggregator; E4 merujuk `problem_set_report.md`. M1–M5 merujuk kumpulan screenshot pada direktori P1–P5, sedangkan K1/K2 merujuk JSON k6 beban dan isolasi pada direktori P2. Seluruh artefak berada di `docs/tex/evidence/`; path singkat seperti `P3/1-new.png` relatif terhadap direktori tersebut, dan path berawalan `evidence/` relatif terhadap `docs/tex/`.

Bukti terdiri atas inspeksi implementasi, hasil tes otomatis tersimpan, tangkapan layar demo, dan pengukuran k6. Bab P1–P5 menyajikan hasil aktual beserta rujukan artefaknya. Hasil k6 mencatat beban 50 VU selama 60 detik dan isolasi latensi BMKG; screenshot RabbitMQ memperlihatkan fanout serta penumpukan dan pengurasan backlog consumer. Status PASS pada suite otomatis dilaporkan sebagai hasil suite dengan cakupan assertion yang dijalankan, bukan pengganti setiap observasi manual. Bukti browser Auth historis dibedakan dari pengujian token API terbaru. Gambar yang menampilkan nilai secret konfigurasi tidak direproduksi dalam PDF.

Bab 2

P1: Arsitektur dan Interoperabilitas Data

2.1 Rancangan Solusi

2.1.1 Kontrak Sumber dan Akuisisi Data

BMKG menyediakan `GET/seismic-events` dan `GET/tsunami-warnings`; PVMBG menyediakan `GET/volcanic-reports`. Ketiganya menerima `since` ISO 8601 yang inklusif dan mengembalikan koleksi `items`. Aggregator tidak meminta sumber mengikuti satu skema baru. Adapter per sumber memahami format asli, mengambil data dengan HTTP, lalu mengubahnya menjadi `HazardEvent`. Kontrak sumber didokumentasikan dalam `contracts/openapi/bmkg.yaml` dan `contracts/openapi/pvmbg.yaml`; kontrak keluaran publik ada pada `contracts/openapi/gateway.yaml`.

Kedua mock menggunakan SQLite file dengan kolom ID, waktu terindeks, dan payload JSON. Inisialisasi membuat tabel serta minimal 20 record ketika penyimpanan belum berisi data; seed tidak ditambahkan ulang pada setiap restart. Generator menambah satu event per interval, default 10 detik. BMKG memakai pola variasi berdasarkan indeks dan UUID untuk ID baru; tidak seluruh nilai gempa dibangkitkan acak. PVMBG memilih variasi laporan dengan bilangan acak, termasuk `confidence_level` ketika skema v2 aktif. Delay BMKG tetap, default 100 ms; delay PVMBG diambil acak antara 500–3000 ms. Sumber implementasi adalah `app.py`, `config.py`, dan `storage.py` pada direktori masing-masing mock.

2.1.2 Model Kanonik dan Pemetaan

`HazardEvent` memisahkan atribut lintas sumber dari rincian mentah. Tujuh field ringkasan adalah `hazard_id`, `source`, `hazard_type`, `severity`, `area_name`, `occurred_at`, dan `ingested_at`. Tampilan penuh menambahkan `source_ref_id`, `latitude`, `longitude`, serta `attributes`. BNPB memberikan ID kanonik dan waktu ingest saat menyimpan record. Unique key pasangan sumber dan ID asli menjaga identitas kanonik pada pengambilan berulang.

Field kanonik	BMKG	PVMBG
`source`	BMKG	PVMBG
`source_ref_id`	`event_id`	`report_id`
`hazard_type`	SEISMIC	VOLCANIC
`area_name`	`region_name`	Nama gunung dari referensi BNPB berdasarkan `volcano_id`.
Koordinat	`epicenter_lat`, `epicenter_lon`	Koordinat referensi gunung statis.
`occurred_at`	`occurred_at`	`reported_at`
`severity`	Tingkat warning terkait; jika tidak ada, berdasarkan magnitudo.	Normal/Waspada/Siaga/Awas menjadi enum huruf kapital.
`attributes`	Magnitudo, kedalaman, potensi tsunami, daftar warning, dan tambahan sumber.	Jumlah erupsi, tinggi kolom abu, ID gunung, confidence bila ada, dan tambahan sumber.

Tabel 2.1: Pemetaan ke model kanonik pada adapter Aggregator.

`BMKGPoller` mengambil gempa serta warning melalui endpoint terpisah, lalu membentuk indeks warning menurut `related_event_id`. Warning dihubungkan ke `event_id` sebelum normalisasi. Jika terdapat lebih dari satu warning, tingkat terberat dipilih dengan urutan AWAS, SIAGA, WASPADA. Tanpa warning, magnitudo $m < 5{,}0$ menghasilkan NORMAL, $5{,}0 \leq m < 6{,}5$ WASPADA, dan $m \geq 6{,}5$ SIAGA. Rincian warning disimpan sebagai array `attributes.tsunami_warnings`, sehingga informasi korelasi tetap tersedia pada proyeksi penuh.

Pemetaan PVMBG memakai `LookupVolcano` di `services/aggregator/internal/domain/volcano.go`. Implementasi menormalkan ID ke huruf kecil dan menerima awalan `v-`. Referensi memuat nama serta koordinat; dengan demikian mock tidak harus menambahkan field lokasi ke kontrak aslinya. Implementasi pemetaan kedua sumber berada di `services/aggregator/internal/ingest/mapper.go`.

2.1.3 Perubahan Skema saat Runtime

Operator mengirim `POST/admin/schema-version` dengan `{"version":2}` pada PVMBG. Pengaturan ini memengaruhi laporan yang dibuat selanjutnya, tanpa restart. Laporan lama tetap tidak memiliki `confidence_level`; laporan baru menyimpan nilai tersebut dalam JSON SQLite. Mengembalikan versi menjadi 1 tidak menghapus field yang telah tersimpan pada laporan v2.

Decoder Aggregator memiliki field `ConfidenceLevel` opsional sekaligus map `RawFields`. JSON dibaca menjadi struktur bertipe untuk field inti dan map untuk field tambahan. Field yang sudah dikenal dikeluarkan dari map, sedangkan field tidak dikenal diteruskan ke `attributes`. Kebijakan ini berlaku pada gempa, warning tsunami, dan laporan vulkanik. Nilai confidence yang tidak ada dibedakan dari nilai numerik 0. PostgreSQL menyimpan atribut dalam JSONB tanpa menambah kolom; kontrak internal membawa atribut melalui struktur dinamis Protobuf. Pemanfaatan JSONB ini sesuai kemampuan penyimpanan dan pemrosesan JSON pada PostgreSQL [6].

2.2 Justifikasi dan Alternatif

Model kanonik menghindarkan klien dari keharusan memahami dua format, dua kredensial sumber, serta aturan korelasi tsunami. Adapter terpisah membuat perubahan pada satu sumber tidak harus merombak API publik. Alternatif meneruskan payload mentah apa adanya lebih

singkat untuk dibuat, tetapi setiap klien harus menduplikasi pemetaan dan pemisahan akses Media.

Struktur inti bertipe dengan map tambahan dipilih sebagai kompromi. Deserialisasi sangat ketat yang menolak semua field baru dapat menghentikan ingest ketika confidence mulai muncul. Sebaliknya, map bebas untuk seluruh record memindahkan kesalahan tipe dan field wajib ke tahap yang lebih jauh. Implementasi saat ini mempertahankan tipe inti dan menggunakan map untuk perubahan aditif. Field tambahan dapat ditelusuri kembali dalam proyeksi penuh, tanpa memperluas tujuh field ringkasan Media.

Polling mengikuti spesifikasi dan memudahkan pengujian sumber yang tidak menyediakan push. Watermark mengurangi pengambilan data berulang, sementara upsert melindungi identitas record. Konsekuensinya, data yang terlihat klien tertinggal setidaknya selama waktu tunggu polling dan pemrosesan; sistem ini bukan integrasi real-time dengan jaminan keterlambatan nol.

2.3 Batasan Solusi

- Dukungan perubahan aditif tidak otomatis menangani rename, penghapusan field wajib, atau perubahan tipe, misalnya `magnitude` menjadi objek. Perubahan semantik tersebut memerlukan pembaruan adapter dan kontrak.
- Field yang tidak dikenal diteruskan, tetapi tidak otomatis dipahami secara bisnis. Penambahan nama yang bertabrakan dengan key atribut hasil normalisasi perlu kebijakan validasi yang lebih tegas.
- ID gunung yang tidak ditemukan menggunakan ID asli sebagai nama area dan koordinat 0,0. Nilai ini merupakan fallback implementasi, bukan koordinat valid yang sudah diverifikasi.
- Fetch warning BMKG dilakukan setelah fetch gempa. Error warning saat ini diabaikan, sehingga kejadian dapat dipetakan berdasarkan magnitudo tanpa pengayaan tsunami. Pembaruan warning untuk gempa yang sudah melewati watermark juga belum mempunyai alur pembaruan mandiri.
- JSONB tidak menghilangkan kebutuhan validasi domain. Data tidak lengkap dan enum tidak dikenal belum memiliki karantina ingest yang menyeluruh.

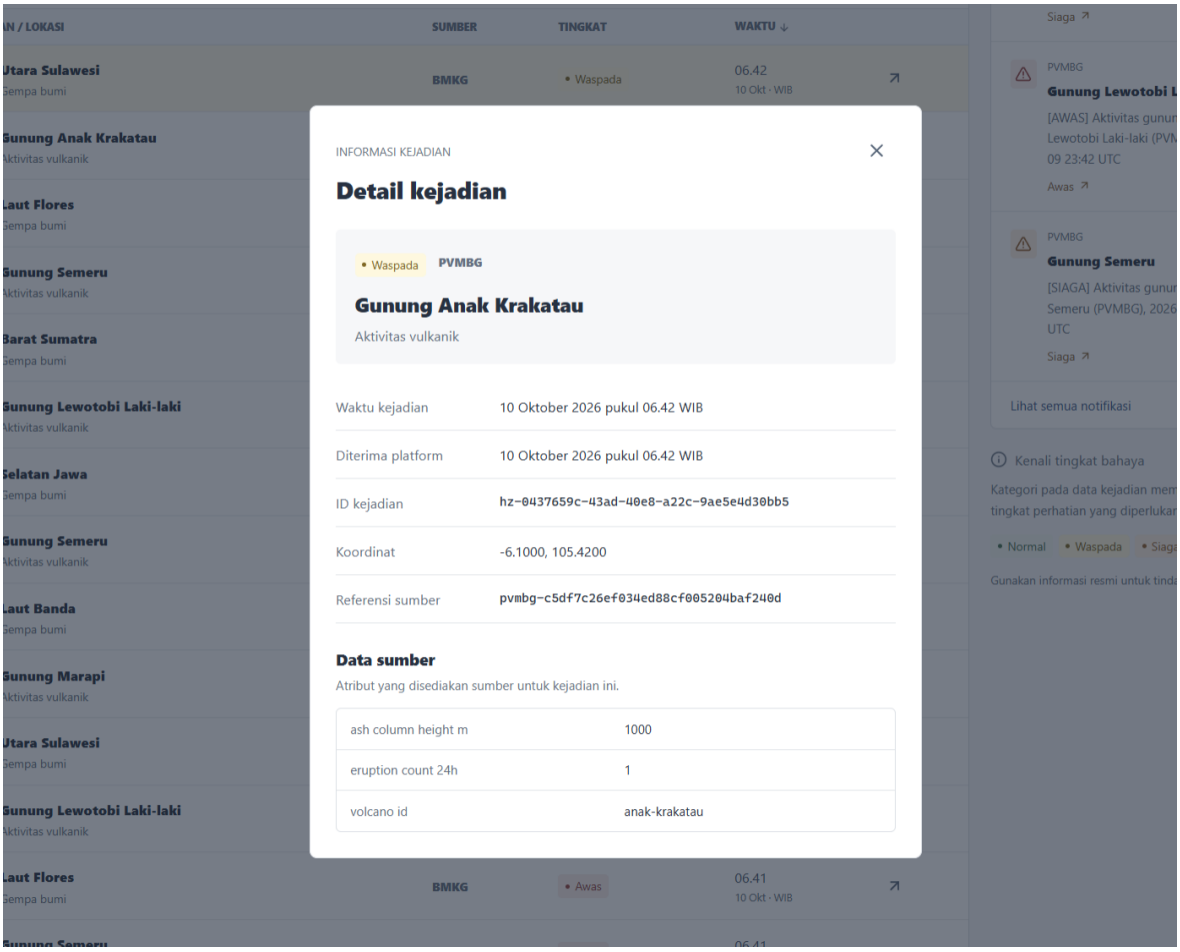
2.4 Proof-of-Concept dan Keterlacakan

Bukti P1 terdiri atas tes unit E3, hasil pengujian stack E4, dan demonstrasi manual M1 yang diuraikan pada bagian ini. E3 merekam tujuh tes tingkat utama paket ingest yang lulus pada 9 Oktober 2026, mencakup fetch HTTP, pemetaan gempa/warning, laporan vulkanik, evolusi skema, serta proyeksi. Paket domain tidak mempunyai test case dan tidak dihitung sebagai tes lulus. E4, yang dibuat pada 10 Oktober 2026, mencatat lima tes P1 lulus, tanpa error/failure maupun skip, dengan durasi 2,42 detik.

2.4.1 Hasil Demonstrasi Manual

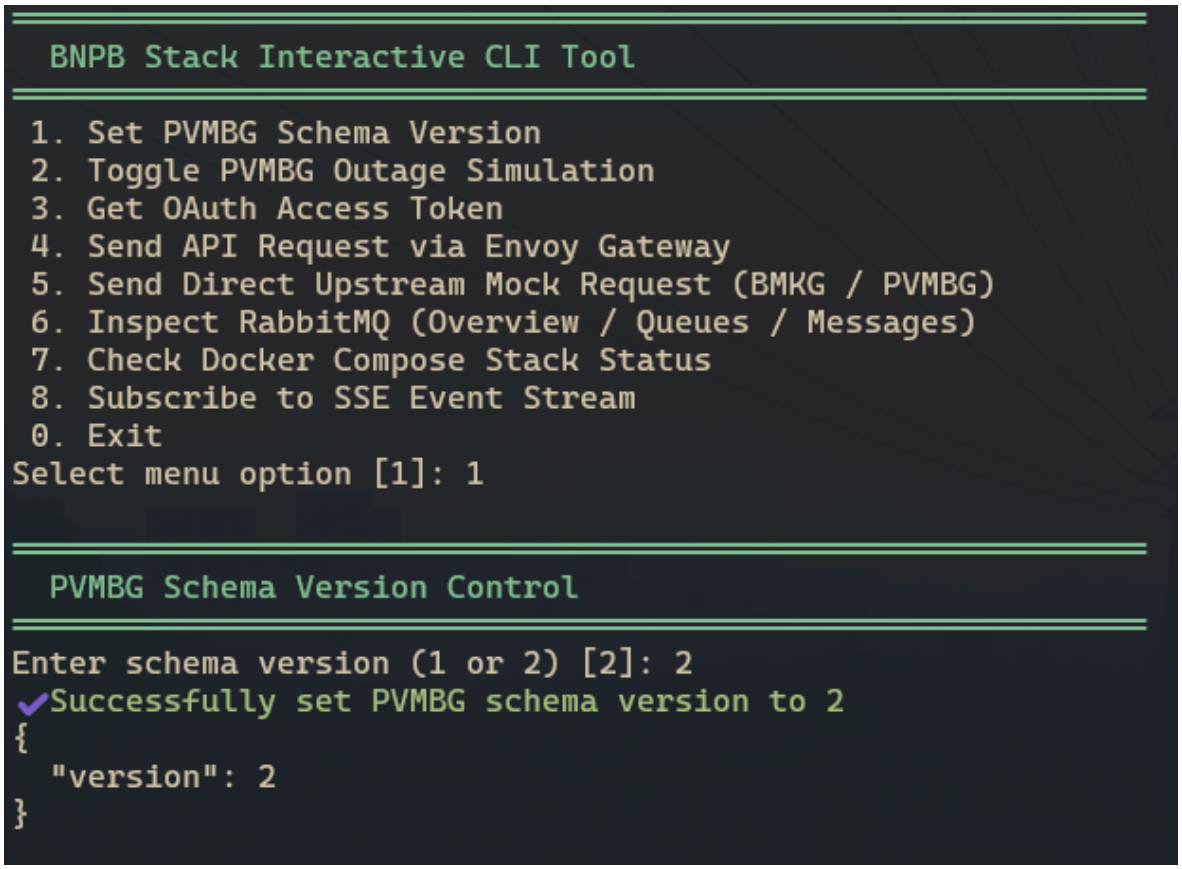
Pada kondisi awal, M1 memperlihatkan detail BMKG Selatan Jawa dengan magnitudo 5,6, kedalaman 20 km, dan severity WASPADA. Detail PVMBG Gunung Anak Krakatau menampilkan ID kanonik, referensi sumber, koordinat, jumlah erupsi, serta tinggi kolom abu tanpa atribut confidence. Kedua tampilan menggunakan bentuk detail kejadian yang sama meskipun rincian sumbernya berbeda. Nilai magnitudo dan severity BMKG pada contoh ini konsisten dengan aturan pemetaan ketika tidak terdapat warning tsunami terkait.

Operator kemudian mengubah versi PVMBG menjadi 2 melalui menu kontrol yang memanggil endpoint admin; respons yang direkam adalah `{"version": 2}`. Sesudah perubahan, antar-

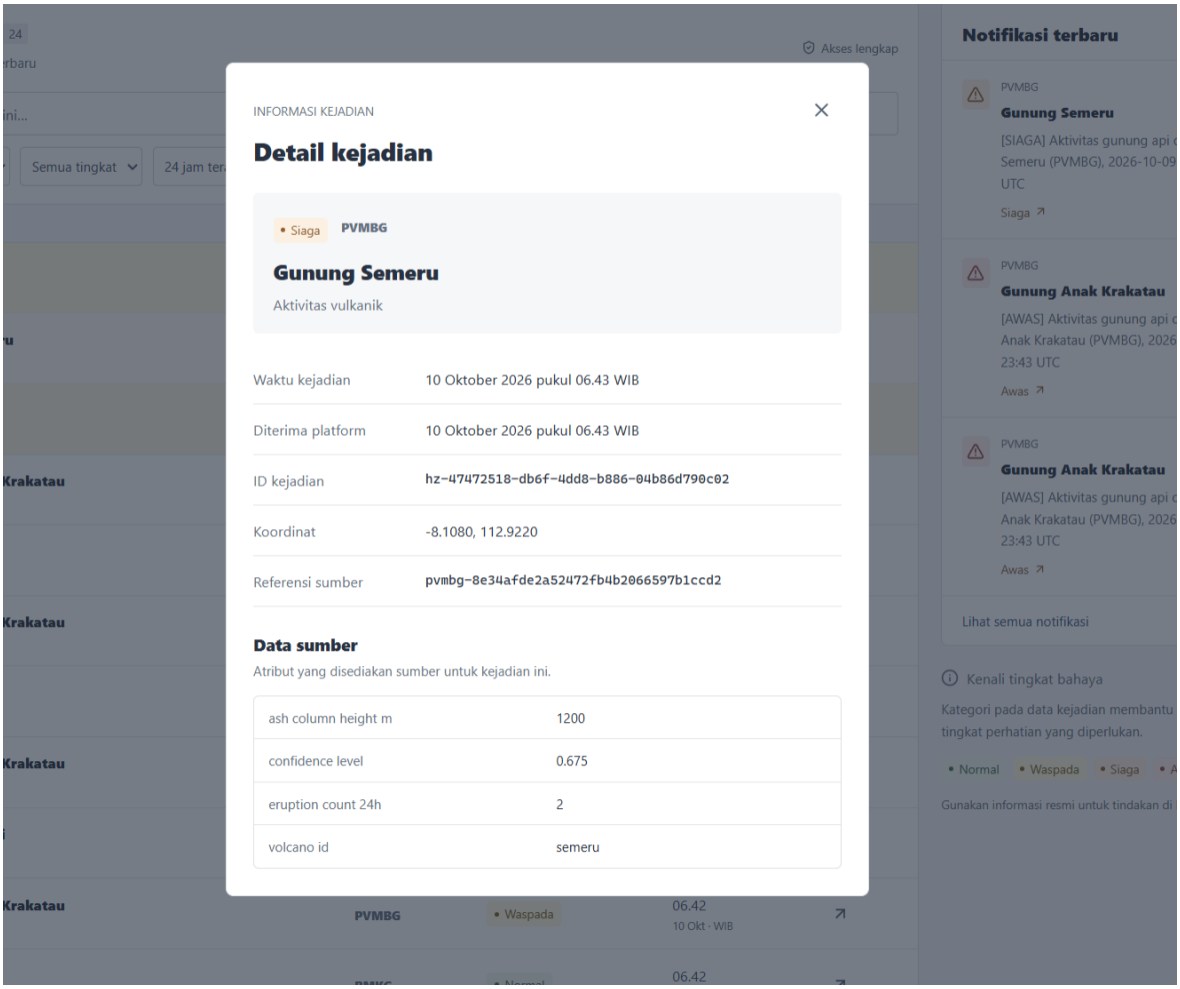


Gambar 2.1: M1: detail PVMBG sebelum perubahan skema; atribut sumber belum memuat confidence.

muka tetap menampilkan kejadian dari kedua sumber. Detail BMKG Barat Sumatra masih memuat magnitudo 5,6 dan severity WASPADA; detail PVMBG Gunung Semeru pada pukul 06.43 WIB tanggal 10 Oktober 2026 memuat `confidence_level` bernilai 0,675, selain atribut vulkanik yang sebelumnya sudah ada. Urutan ini memperlihatkan bahwa field tambahan diteruskan sampai tampilan klien BNPB.



Gambar 2.2: M1: respons kontrol runtime ketika versi skema PVMBG diubah menjadi 2.



Gambar 2.3: M1: detail PVMBG sesudah perubahan; confidence bernilai 0,675 tampil bersama atribut vulkanik lainnya.

2.4.2 Cakupan Bukti

Kriteria	Bukti tersedia	Kesimpulan dan batas cakupan
Pemetaan dua sumber	E3; E4 tes awal; M1 `P1/1-1.png` dan `P1/1-2.png`.	Struktur kanonik dan contoh nilai kedua sumber terlihat. Tes unit melengkapi pemeriksaan aturan korelasi warning.
Perubahan v1 ke v2	M1 `P1/2.png`, `P1/3-1.png`, dan `P1/3-2.png`; E4 tes perubahan skema.	Kontrol versi berhasil dan kedua sumber tetap tampil; confidence terlihat pada hasil PVMBG.
Field tambahan pada attributes	E3 tes evolusi skema; E4 tes penerusan field; M1 detail v2.	Kebijakan aditif teramati untuk confidence; tidak menyatakan semua jenis perubahan skema didukung.
Koeksistensi record v1/v2	E4 `test_05_v1_and_v2_coexistence_in_canonical_store`.	Tes memeriksa record tanpa dan dengan confidence dalam hasil query yang sama. Screenshot M1 menunjukkan contoh sebelum dan sesudah secara terpisah.
Kontinuitas tanpa restart	Kode kontrol runtime dan urutan M1.	Artefak tidak menyertakan pasangan ID/start time container sebelum–sesudah; kontinuitas proses belum diukur langsung.

Tabel 2.2: Keterlacakan P1 terhadap bukti unit, otomatis, dan manual.

Hasil E4 memperkuat demonstrasi manual, dengan batas metode bahwa pencarian record v2 pada tes menerima record yang sudah memiliki confidence dan tidak mengikatnya pada ID serta waktu sesudah toggle. Oleh karena itu, laporan membedakan keberhasilan membaca data v1/v2 dari pembuktian bahwa record tertentu baru dihasilkan pada percobaan tersebut. Respons mentah sebelum–sesudah serta snapshot container akan memperkuat keterlacakan pada pengujian berikutnya; bukti M1 yang tersedia sudah memperlihatkan hasil perubahan aditif pada antarmuka.

Bab 3

P2: Konkurensi dan Ketersediaan

3.1 Rancangan Solusi

Sistem memisahkan pengambilan data sumber dari pelayanan permintaan pengguna. Aggregator menjalankan poller BMKG dan PVMBG sebagai dua goroutine independen, sementara layanan gRPC membaca data kanonis di PostgreSQL. Publisher outbox berjalan pada goroutine lain. Pemisahan ini terlihat pada `services/aggregator/cmd/main.go`. Dengan demikian, keterlambatan PVMBG memengaruhi waktu pembaruan data PVMBG, bukan menambahkan waktu tunggu upstream ke setiap pembacaan BMKG. Permintaan pengguna tetap bergantung pada kapasitas gateway, autentikasi, Aggregator, dan database yang dipakai bersama.

Setiap poller melakukan pengambilan awal, kemudian mengulangi polling menurut interval masing-masing. HTTP client memiliki timeout per sumber dan menerima context untuk pembatalan saat layanan berhenti. Dalam satu poller, satu putaran diselesaikan sebelum putaran berikutnya diproses; implementasi tidak membuat goroutine baru tanpa batas pada setiap tick. BMKG mengambil seismic event lalu peringatan tsunami secara berurutan, sedangkan PVMBG mengambil laporan vulkanik. Implementasi berada pada `services/aggregator/internal/ingest/bmkg.go` dan `services/aggregator/internal/ingest/pvmbg.go`.

Tabel 3.1: Parameter konkurensi dan kesegaran data

Parameter	**Nilai acuan**	**Makna**
Interval polling kedua sumber	3.000 ms	Contoh konfigurasi di `.env.example`.
Timeout BMKG / PVMBG	1.000 / 3.500 ms	Batas setiap request HTTP upstream, bukan target latensi API.
Ambang stale	10.000 ms	Acuan usia keberhasilan pengambilan sumber.
Timeout unary gateway	2.000 ms	Default deadline panggilan backend.
Permintaan unary aktif	100	Default kapasitas semaphore gateway; stream dikecualikan.
Koneksi PostgreSQL	5–25	Minimum dan maksimum pool Aggregator.

Setiap sumber mempunyai watermark, waktu keberhasilan terakhir, waktu percobaan terakhir, dan pesan kegagalan. Setelah fetch berhasil, data dinormalisasi dan disimpan sebelum watermark dimajukan. Pada kegagalan, data sebelumnya tidak dihapus dan watermark keberhasilan dipertahankan. Pada pembacaan API, `services/aggregator/internal/domain/status.go` menghitung status berikut:

- **FRESH**: keberhasilan terakhir masih dalam ambang waktu dan tidak ada error sumber yang

tercatat.
- **STALE**: sumber gagal atau terlalu lama tidak berhasil, tetapi data kanonis sumber tersebut masih tersedia.
- **UNAVAILABLE**: tidak tersedia data tersimpan untuk digunakan ketika sumber belum berhasil atau tidak dapat diperbarui.

Respons hazard tetap membawa item yang tersedia beserta metadata `freshness`. Nilai `partial=true` diberikan apabila sumber yang relevan dengan permintaan tidak fresh. Filter sumber membatasi pemeriksaan kesegaran pada sumber tersebut. Mekanisme ini membedakan kondisi tidak ada peristiwa dari sumber yang belum dapat memberikan data, sekaligus memungkinkan klien menampilkan usia data. Pembentukan respons ada pada `services/aggregator/internal/grpc/server.go`; penyimpanan watermark ada pada `services/aggregator/internal/store/hazard_store.go`.

Gateway juga membatasi permintaan unary aktif dengan channel berkapasitas tetap. Ketika penuh, permintaan ditolak dengan respons overload sesuai kontrak, umumnya HTTP 429, dan `Retry-After`. Deadline unary membatasi waktu tunggu backend. Pembatas ini bukan pembatas laju per pengguna dan tidak menghitung stream SSE yang berumur panjang. Kode terkait adalah `services/api-gateway/internal/server/server.go`, `services/api-gateway/internal/config/config.go`, dan `services/api-gateway/internal/problem/problem.go`.

3.2 Justifikasi dan Alternatif

Polling independen dipilih karena data kebencanaan dikumpulkan secara berkala dan dapat dilayani dari penyimpanan kanonis. Pengguna memperoleh respons dari satu bentuk data meskipun kedua sumber memiliki latensi serta skema berbeda. Biaya keputusan ini adalah jeda antara perubahan upstream dan kemunculannya pada API. Karena itu, freshness merupakan bagian dari kontrak respons, bukan sekadar informasi log.

Alternatif pertama ialah melakukan dua panggilan upstream secara serial pada setiap request. Pendekatan tersebut sederhana, tetapi latensi sumber terakumulasi dan gangguan satu sumber langsung memengaruhi request pengguna. Alternatif kedua ialah fan-out paralel per request dengan deadline terpisah. Pendekatan kedua mengurangi penjumlahan waktu tunggu, tetapi tetap menambah beban upstream sebanding dengan jumlah pengguna dan memerlukan fallback tersendiri.

Penyimpanan kanonis dengan polling mengurangi keterikatan tersebut, dengan konsekuensi kapasitas database menjadi sumber daya bersama. Pool koneksi dan batas unary gateway menahan pertumbuhan pekerjaan aktif. Penggunaan circuit breaker atau penjadwal polling terdistribusi dapat menjadi pengembangan berikutnya, tetapi keduanya belum menjadi mekanisme yang digunakan pada implementasi ini.

3.3 Batasan Solusi

Isolasi pada tingkat goroutine tidak menjamin isolasi CPU, memori, database, atau autentikasi. Permintaan BMKG dan PVMBG tetap melewati komponen bersama. Kegagalan PostgreSQL tidak dapat ditutupi oleh fallback memori karena jalur baca kanonis masih membutuhkan database. Compose juga belum memberikan bukti failover beberapa replika Aggregator atau database.

Status kesegaran BMKG terutama mengikuti keberhasilan seismic event. Error pengambilan peringatan tsunami saat ini diabaikan oleh poller, sehingga kegagalan pengayaan tsunami belum selalu muncul sebagai status stale. Selain itu, pembaruan watermark dilakukan sesudah

transaksi data selesai, bukan di dalam transaksi yang sama. Restart pada sela kedua operasi dapat menyebabkan data diambil kembali; kestabilan identitas hazard membantu menjaga baris kanonis, tetapi tidak membuktikan tidak ada publication berulang.

Nilai timeout, kapasitas pool, serta semaphore merupakan konfigurasi atau mekanisme yang ditemukan pada kode. Hasil beban yang dilampirkan di bawah mengukur satu lingkungan lokal dan satu jendela pengujian; hasil tersebut tidak merupakan jaminan kapasitas produksi atau availability jangka panjang. Skenario outage membuktikan degradasi dan pemulihan pada demonstrasi yang dicatat, bukan failover infrastruktur.

3.4 Proof-of-Concept dan Keterlacakan

Bukti historis container pada 30 September 2026 disimpan dalam `docs/tex/evidence/mock-container-2026-09-30.json`. Catatan tersebut memuat 10 pemeriksaan berhasil pada lingkup mock. Pengukuran langsung BMKG ketika empat request PVMBG berjalan paralel tercatat 113,86 ms; pemeriksaan health dan admin PVMBG tercatat masing-masing 22,29 ms dan 17,80 ms. Angka ini menunjukkan perilaku pada skenario mock yang dicatat, bukan p95 gateway atau hasil 50 pengguna selama 60 detik.

Tabel 3.2: Status pembuktian P2 berdasarkan bukti yang dilampirkan

Skenario	**Target atau cakupan**	**Hasil**
Konkurensi mock	Request BMKG bersamaan dengan empat request PVMBG	Bukti historis tersedia.
Beban API lengkap	50 VU selama 60 detik; error tidak terkendali di bawah 1%	Terpenuhi: error 0%, p95 243,96 ms (K1, M2).
Isolasi melalui gateway	Target delay PVMBG 3 detik; p95 grup BMKG di bawah 300 ms	p95 87,09 ms; probe delay aktual sekitar 2,92 detik (K2, M2).
Degradasi sumber	Data tersimpan tetap dilayani dan status kembali fresh sesudah outage dinonaktifkan	HTTP 200, PVMBG stale, lalu pulih (M2, E4).

Repositori menyediakan skenario yang dapat diulang pada `tests/load/aggregator.js` dan `tests/load/aggregator-isolation.js`, dengan petunjuk pada `docs/load-testing.md`. Konfigurasi pengujian dan prosedur pengulangannya adalah:

1. Jalankan stack yang telah diprovisikan, pastikan akun layanan tersedia pada `.cache/auth/`, dan buat direktori `.cache/load/`. Jalankan `k6runtests/load/aggregator.js` dari root repositori. Defaultnya adalah pemanasan 10 detik, lalu 50 VU selama 60 detik. Campuran request adalah 60% daftar, 30% detail, dan 10% status sumber. Hanya fase beban dihitung; kegagalan pengambilan token dicatat terpisah.
2. Untuk isolasi, tetapkan `PVMBG_RESPONSE_DELAY_MIN_MS` dan `PVMBG_RESPONSE_DELAY_MAX_MS` menjadi 3000, lalu buat ulang `pvmbg-mock` sebagaimana panduan. Jalankan `k6runtests/load/aggregator-isolation.js`. Setup memeriksa tiga request langsung ke mock dan membatalkan pengujian bila median delay kurang dari 2900 ms.
3. Bandingkan fase baseline 30 detik dengan fase kontensi 60 detik, setelah pemanasan 5 detik. Kedua fase menggunakan 10 VU BMKG yang diukur dan 10 VU kelompok kedua; kelompok kedua beralih dari BMKG ke PVMBG. Simpan p50/p95/p99, error, throughput, ringkasan JSON, dan jendela waktu untuk pencocokan trace. Pulihkan konfigurasi delay setelah selesai.

Latensi yang diukur skrip tersebut mencakup Envoy, token exchange, gateway, RPC, dan pembacaan database. Angka yang dilaporkan merupakan latensi jalur API lengkap, bukan waktu eksekusi Aggregator saja. Bukti K1 berada pada `evidence/P2/aggregator-2026-10-09T15-31-17-502Z.json`; bukti K2 berada pada `evidence/P2/aggregator-isolation-2026-10-09T15-37-25-357Z.json`, relatif terhadap direktori laporan. Tangkapan layar M2 memperlihatkan konfigurasi, fase, hasil, dan nama ringkasan JSON yang bersesuaian.

Untuk menilai batas error spesifikasi, HTTP 429 yang merupakan backpressure terkendali dilaporkan terpisah dari error tidak terkendali. Pada fase beban, catat jumlah request API N, jumlah 429 N_{429}, dan kegagalan lain N_{err}; laporkan $100N_{\text{err}}/N$ serta $100N_{429}/N$. Denominator dan definisi kegagalan harus dicantumkan agar angka dapat diperiksa. Threshold error bawaan skrip masih menghitung 429, sehingga lebih ketat daripada penghitungan penerimaan yang mengecualikannya. Hasil token endpoint tidak dicampur ke persentil request API.

3.4.1 Beban Berkelanjutan dan Isolasi Latensi

Pengujian K1 pada 9 Oktober 2026 menjalankan 50 VU selama 60 detik setelah pemanasan 10 detik. Fase beban mencatat 17.546 request API: 10.501 daftar, 5.272 detail, dan 1.773 status sumber. Tidak ada kegagalan API, HTTP 429, atau kegagalan pengambilan token pada fase tersebut. Throughput fase beban adalah $17\,546/60 = 292{,}43$ request/detik. Nilai `rate` bawaan pada ringkasan JSON menggunakan durasi seluruh run, termasuk pemanasan, sehingga tidak digunakan sebagai throughput jendela beban 60 detik.

Tabel 3.3: Hasil pengukuran P2; latensi dalam milidetik

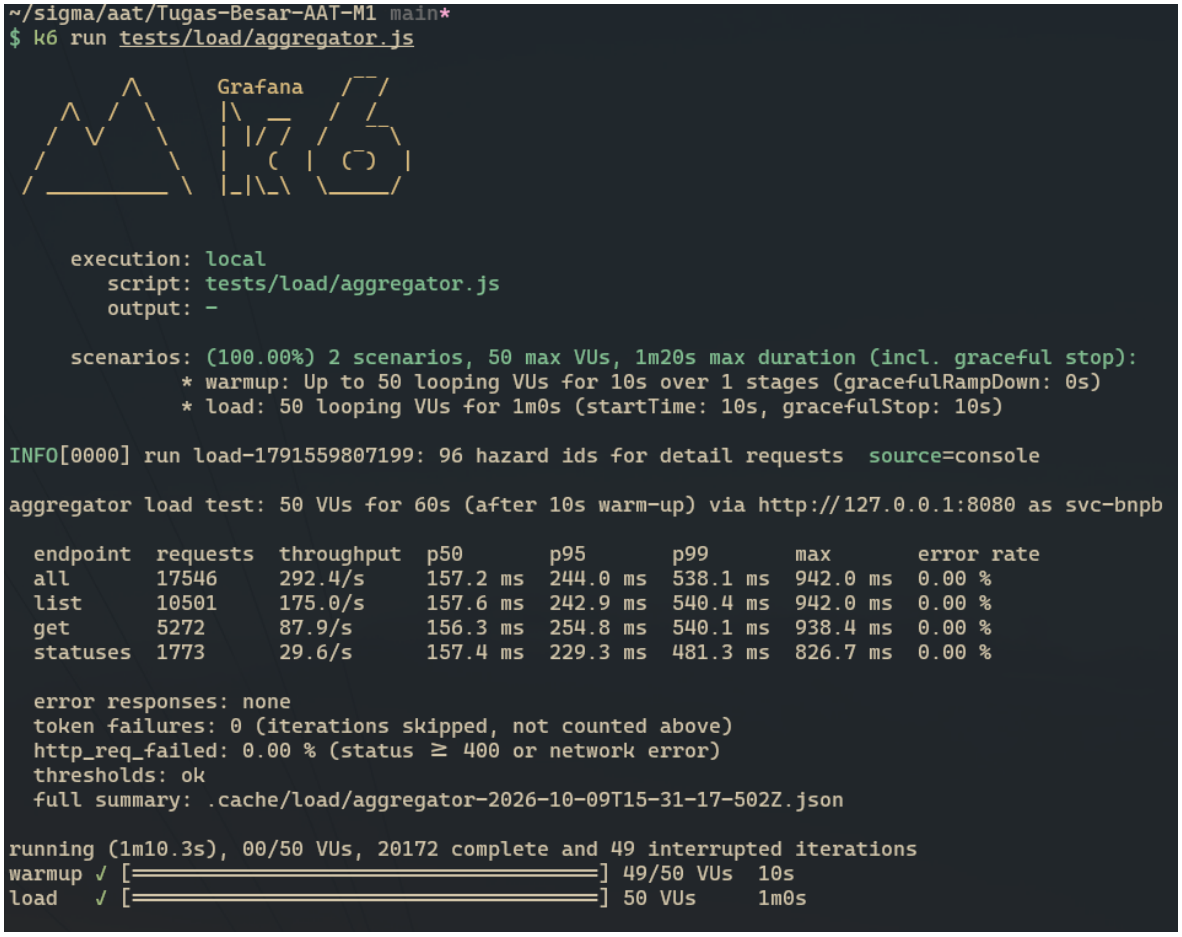
Metrik	**Beban 50 VU / 60 detik (K1)**	**BMKG saat kontensi / 60 detik (K2)**
Jumlah request	17.546	9.407
Throughput (request/detik)	292,43	156,78
p50	157,19	59,50
p95	243,96	87,09
p99	538,14	126,99
Error tidak terkendali	0 / 17.546 (0%)	0 / 9.407 (0%)
HTTP 429	0	0

Pengujian K2 menggunakan 10 VU BMKG yang diukur dan 10 VU kelompok kedua. Pada baseline 30 detik, kelompok kedua juga meminta BMKG; pada fase kontensi 60 detik, kelompok kedua meminta PVMBG. Probe langsung ke mock mencatat delay 2.921,29; 2.921,80; dan 2.928,30 ms. Nilai aktual ini memenuhi ambang validasi skrip 2.900 ms dan mendekati target konfigurasi 3.000 ms; hasil pengukuran tidak dinyatakan tepat 3.000 ms.

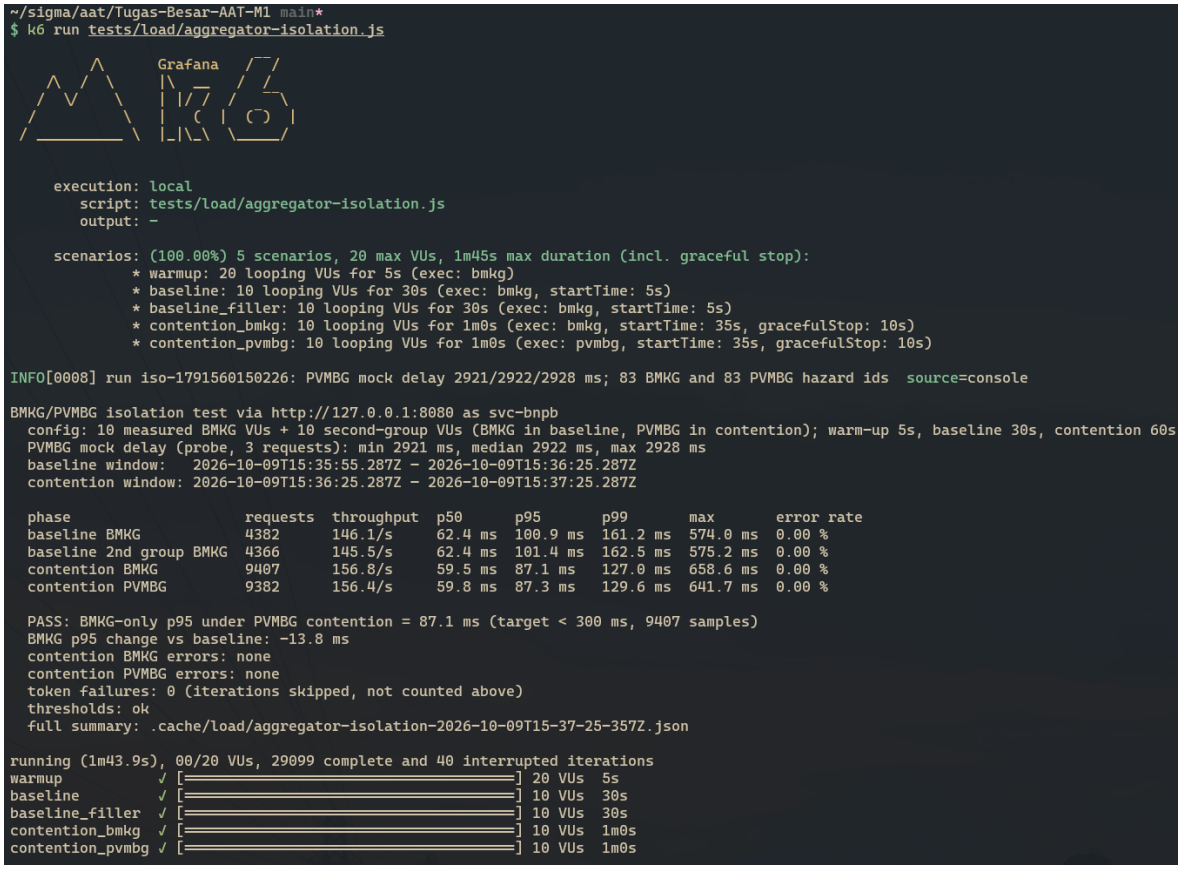
Pada baseline, p95 kelompok BMKG adalah 100,87 ms. Pada fase kontensi, p95 BMKG menjadi 87,09 ms dari 9.407 request, memenuhi ambang 300 ms; kelompok PVMBG menjalankan 9.382 request tanpa error. Kedua fase melewati jalur API publik yang sama. Perbedaan p95 sebesar sekitar $-13{,}78$ ms merupakan hasil run ini dan tidak ditafsirkan sebagai jaminan bahwa tambahan beban PVMBG selalu mempercepat BMKG.

3.4.2 Outage, Data Stale, dan Pemulihan

Urutan bukti M2 terdapat pada `evidence/P2/3-1.png` sampai `evidence/P2/3-5.png`. Outage PVMBG diaktifkan melalui kontrol admin, kemudian dashboard memperlihatkan BMKG tetap terkini dan PVMBG terlambat. Respons `GET/api/v1/hazards` melalui Envoy tetap HTTP 200 dengan metadata BMKG `FRESH` dan PVMBG `STALE`. Respons juga menampilkan `last_succe`

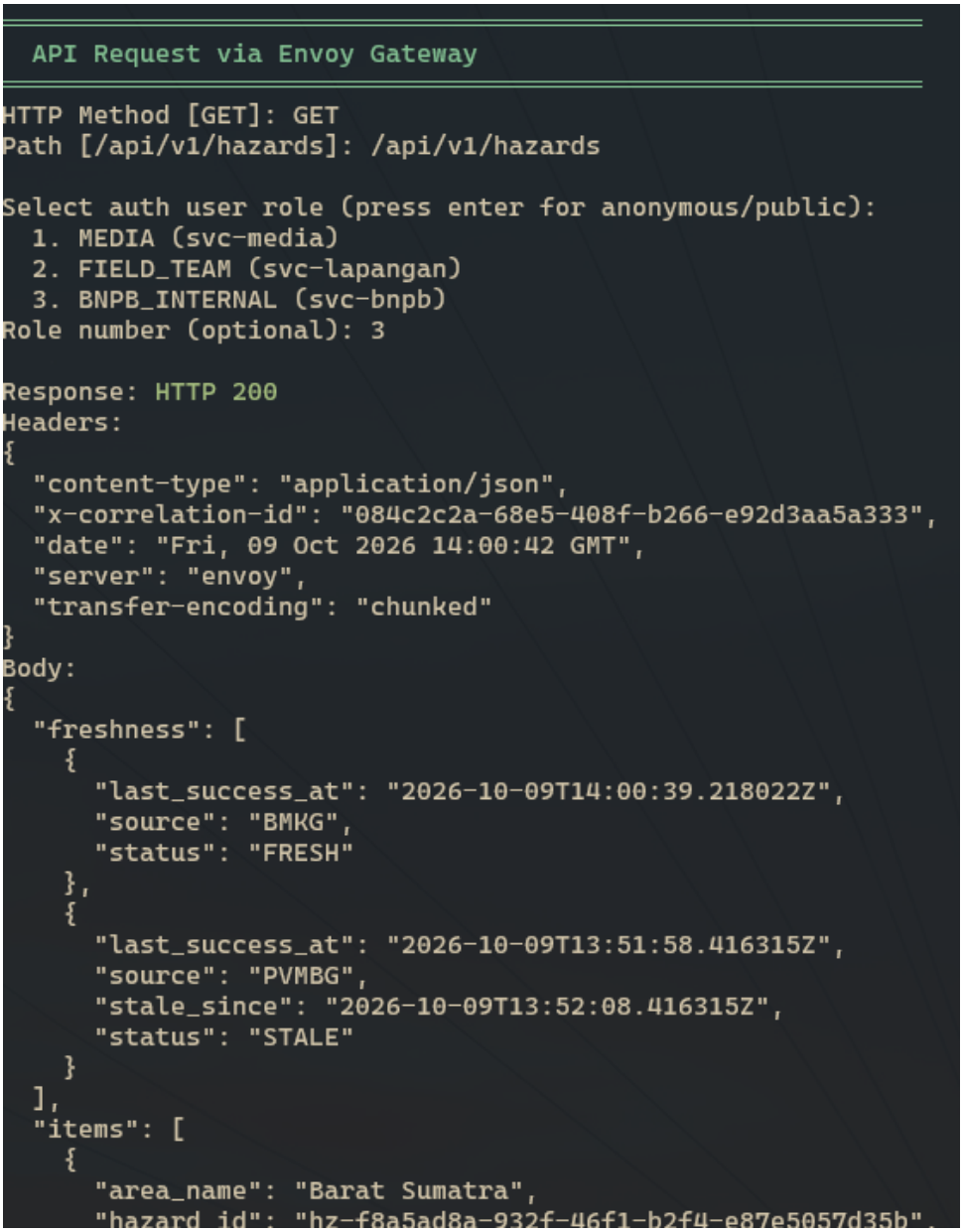


Gambar 3.1: Hasil k6: pemanasan 10 detik diikuti 50 VU selama 60 detik; fase beban mencatat 17.546 request tanpa error (M2, K1).



Gambar 3.2: Hasil isolasi: probe PVMBG sekitar 2,92 detik dan p95 BMKG 87,09 ms saat kontensi, di bawah target 300 ms (M2, K2).

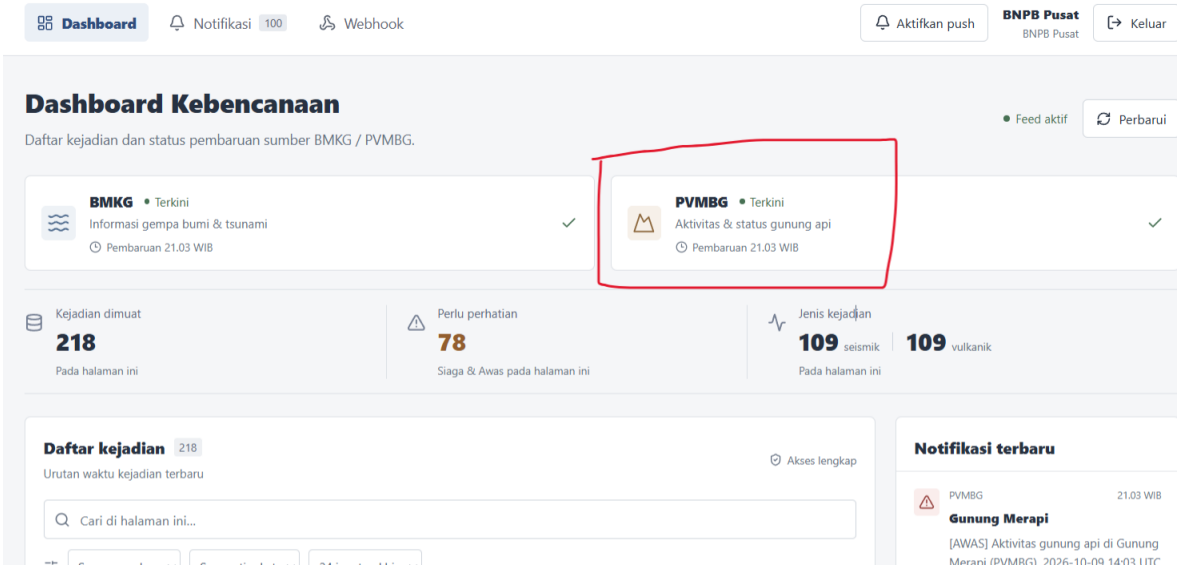
`ss_at` dan `stale_since` untuk PVMBG, sehingga gangguan sumber tidak ditampilkan sebagai tidak adanya kejadian. Setelah outage dinonaktifkan, dashboard kembali memperlihatkan kedua sumber terkini.



Gambar 3.3: Respons API HTTP 200 saat outage PVMBG: BMKG tetap fresh, sedangkan PVMBG ditandai stale beserta waktu kesegarannya (M2).

Ringkasan otomatis E4, `evidence/problem_set_report.md`, mencatat tiga tes P2 berhasil dalam 74,97 detik. Tes outage pada `tests/test_problem_2.py` memeriksa data PVMBG tersimpan tetap tersedia, HTTP 200, status stale, serta pemulihan ke fresh setelah outage dinonaktifkan; prosedur tes tersebut tidak menjalankan restart. Namun, gambar manual tidak melampirkan identitas dan waktu mulai container sebelum/sesudah, sehingga gambar saja tidak membuktikan tidak terjadi restart di luar prosedur.

Narasi kriteria pada E4 merupakan template ringkasan, bukan rekaman angka latensi atau konfigurasi efektif. Pengukuran 50 VU/60 detik dan validasi delay PVMBG karena itu dirujukkan ke K1 dan K2. Bukti ini mendukung skenario yang terekam, tanpa mengklaim ketahanan outage berkepanjangan atau availability di luar jendela pengujian.



Gambar 3.4: Dashboard sesudah outage dinonaktifkan memperlihatkan BMKG dan PVMBG kembali terkini (M2).

Bab 4

P3: Autentikasi, Otorisasi, dan Trust

4.1 Rancangan Solusi

4.1.1 Pemisahan Domain Kepercayaan

BMKG menerima API key melalui `X-BMKG-Key`; PVMBG menerima token opaque melalui `Authorization:Bearer`. Keduanya memvalidasi secret milik sendiri. Endpoint data dan kontrol PVMBG terlindungi, sedangkan health dapat dipakai tanpa kredensial. Aggregator mengirim header yang sesuai pada adapter sumber; kredensial pengguna BNPB tidak diteruskan ke mock.

Helper `tools/setup_mock_env.py` membuat dua nilai acak berbeda dengan generator secret, menolak penggunaan nilai sama, dan mempertahankan secret yang sudah valid. Helper hanya memprovisikan konfigurasi; layanan dapat dijalankan tanpa helper bila environment sudah diisi. SQLite menyimpan data mock, bukan menjadi sumber API key. Seorang anggota tim yang menyiapkan environment sendiri memperoleh secret lokal sendiri dan harus menyamakan konfigurasi pemanggil serta penerima pada instalasinya.

Untuk pengguna, Zitadel menjadi identity provider sekaligus authorization server. OIDC memberikan identitas login melalui ID token; OAuth menyediakan access token untuk akses API [3]. Akun `media-demo`, `lapangan-demo`, dan `bnpb-demo` mempunyai subject dan kredensial terpisah di PostgreSQL Zitadel. Bootstrap mengatur role, grant, aplikasi OIDC, serta TTL access token default 60 detik. Akun service untuk tes API juga diprovisikan terpisah. SQLite Auth bridge hanya menyimpan sesi dan transaksi login.

4.1.2 Login, Cookie, dan Pemanggilan API

Alur login menggunakan Authorization Code dengan PKCE S256. PKCE mengikat pertukaran authorization code dengan verifier transaksi, sehingga kepemilikan code saja tidak cukup untuk menukarnya [4].

1. Browser membuka `/auth/login`. Bridge membuat state, nonce, verifier PKCE, dan cookie transaksi, lalu mengarahkan pengguna ke hosted Login UI Zitadel. Password dikirim ke provider.
2. Callback memvalidasi transaksi browser dan menukar authorization code. Library `open id-client` memeriksa ID token, issuer, audience, nonce, serta masa berlaku. Introspeksi access token harus menghasilkan identitas aktif dengan audience project dan role/scope yang dikenal.
3. Bridge menyimpan refresh token serta payload sesi dalam SQLite yang dienkripsi AES-256-GCM. Browser memperoleh cookie `bnpb_access` berisi access token opaque asli dan `bnpb_s ession` berisi ID sesi acak.

4. Kedua cookie bersifat HttpOnly, SameSite=Lax, host-only, dan Path=/; Secure diaktifkan pada HTTPS. Respons `/auth/session` hanya mengembalikan profil, scope, expiry, dan CSRF token. Token akses/refresh tidak dimasukkan ke JSON ataupun penyimpanan JavaScript.
5. Untuk `/api/*`, filter Lua Envoy memindahkan access cookie menjadi header bearer dan menghapus cookie sebelum meneruskan request. Campuran bearer eksplisit dan cookie Auth ditolak agar sumber kredensial tidak ambigu.
6. Filter token exchange Envoy menggunakan confidential client ke Zitadel dan meminta JWT melalui mekanisme RFC 8693 [5]. JWT diteruskan melalui API Gateway sebagai metadata otorisasi gRPC. Service bisnis memverifikasi token dan memeriksa izin method.

Target Rust `zitadel-claims` menerima hook Actions v2 sebelum pembentukan access token dan userinfo/introspeksi. Target memeriksa signature HMAC-SHA256 beserta timestamp panggilan dari Zitadel, lalu membentuk `role` dan `scopes` dari grant pada project/organisasi yang diizinkan. Claim tidak ditentukan oleh input role di frontend. Target ini tidak dibuka sebagai endpoint publik pada Envoy.

4.1.3 Hak Akses dan Pembatasan Data

Hak akses	MEDIA	FIELD_TEAM	BNPB_INTERNAL
Hazard ringkasan	Diizinkan	Diizinkan	Diizinkan
Hazard penuh/mentah	Ditolak	Diizinkan	Diizinkan
Baca notifikasi	Diizinkan	Diizinkan	Diizinkan
Baca webhook	Diizinkan sesuai kepemilikan	Diizinkan sesuai kepemilikan	Diizinkan sesuai kepemilikan
Buat/ubah webhook	Ditolak	Diizinkan	Diizinkan

Tabel 4.1: Scope role yang dibentuk oleh target Actions. Pemeriksaan izin berada pada backend.

Untuk hazard, Aggregator menolak permintaan `VIEW_FULL` tanpa `hazards:read:full`. Proyeksi summary hanya memuat tujuh field ringkasan Bab 2; koordinat, referensi asli, dan attributes tidak disertakan. API Gateway menerjemahkan representasi Protobuf menjadi bentuk OpenAPI dan menolak bentuk backend yang melanggar proyeksi. Menyembunyikan tombol detail pada UI hanya mendukung pengalaman pengguna; pengamanan utama adalah pengecekan scope dan pemilihan proyeksi pada server.

4.1.4 Refresh, Logout, dan CSRF

SPA menjadwalkan refresh sekitar lima detik sebelum expiry serta dapat memulihkan sesi setelah 401. Bridge menggunakan refresh token server dan menggabungkan request refresh bersamaan untuk sesi yang sama. Rotasi data sesi dan pencatatan hash token yang ditolak dilakukan dalam transaksi SQLite. Pengguna tidak perlu memasukkan password lagi selama refresh token dan sesi masih berlaku. Kode bridge juga mencoba mencabut access token lama pada Zitadel setelah rotasi.

POST refresh/logout mensyaratkan Origin aplikasi dan kecocokan nilai `X-CSRF-Token` dengan sesi. Pada mutasi API berbasis cookie, Envoy memeriksa Origin dan keberadaan header CSRF; Envoy tidak melakukan perbandingan dengan SQLite bridge. Kebijakan ini bergantung pada origin yang tepat dan tidak dibukanya CORS untuk origin sembarang. Bearer eksplisit bagi klien API mengikuti jalur tersendiri tanpa cookie browser.

Logout menghapus sesi lokal, membersihkan cookie, dan mencoba revokasi provider. Browser diarahkan ke logout Zitadel untuk mengakhiri SSO. Login UI V2 dapat menampilkan tindakan *End Session*; penghapusan cookie aplikasi saja tidak selalu mengakhiri sesi provider. Implementasi ada di `services/auth-bridge/src/server.mjs`, `provider.mjs`, dan `store.mjs`.

4.2 Justifikasi dan Alternatif

API key/opaque token sederhana dipakai untuk mock agar perbedaan protokol autentikasi antarsumber tetap terlihat tanpa membangun provider untuk setiap instansi. Pemisahan secret membatasi penggunaan kredensial lintas instansi. Konsekuensinya, rotasi dan distribusi secret mock tetap menjadi tugas konfigurasi.

OIDC terpusat menghindari pembuatan form password, hash password, dan penerbitan JWT sendiri di frontend BNPB. Dibanding menyimpan bearer token dalam localStorage, cookie HttpOnly mengurangi keterbacaan token oleh JavaScript, tetapi menambah kebutuhan perlindungan CSRF. Bridge dipakai karena refresh token serta client secret perlu berada di server. Dibanding introspeksi pada setiap service, token exchange di edge memberikan format JWT untuk validasi backend. Biayanya adalah dependensi pada provider pada jalur masuk dan tambahan konfigurasi client.

Alternatif mTLS dapat mengautentikasi identitas workload antarlayanan dengan lebih kuat, tetapi memerlukan pengelolaan sertifikat. Implementasi M1 menggunakan JWT pengguna serta token internal `x-service-token` pada jalur yang mendukungnya. Token internal bukan identitas pengguna dan tidak boleh diberikan kepada browser. Pembagian broker user per consumer akan memberi isolasi lebih baik daripada satu akun broker bersama, tetapi belum diterapkan seragam pada Compose.

4.3 Batasan Solusi

- Revokasi access token lama pada provider dilakukan asinkron dan kegagalannya ditoleransi. Daftar penolakan lokal bridge tidak dibaca Envoy. Karena itu penolakan seketika pada gateway tidak dapat dijamin jika revokasi gagal; token lama mungkin berlaku hingga expiry. JWT hasil exchange yang sudah diterbitkan juga memiliki siklus validasinya sendiri.
- Validasi audience belum seragam: Aggregator memeriksanya bila dikonfigurasi, sedangkan Danger Notification memiliki `ValidateAudience=false`. Verifier Aggregator memberi toleransi waktu 30 detik; ini perlu dibedakan dari expiry pada token opaque yang dicek provider di edge.
- Compose lokal memakai HTTP dan gRPC plaintext, bukan mTLS. Koordinasi refresh dan SQLite sesi ditujukan untuk satu instance bridge; replikasi membutuhkan koordinasi/penyimpanan bersama.
- Secret mock, Auth, PAT, dan file Firebase dikeluarkan dari Git, tetapi Compose canonical database masih menulis password pengembangan literal. Selain itu beberapa service menerima seluruh `.env` dan akun RabbitMQ bersama. Dengan demikian ketentuan tanpa secret hardcoded dan distribusi secret minimum belum terpenuhi menyeluruh; perlu perbaikan konfigurasi sebelum klaim tersebut dinyatakan lulus.
- Provider yang tidak tersedia dapat menyebabkan token exchange gagal meskipun data hazard masih tersimpan. Graceful degradation terhadap mock pada P2 tidak otomatis menghilangkan dependensi Auth ini.

4.4 Proof-of-Concept dan Keterlacakan

Bukti P3 menggabungkan E1 untuk kredensial mock, E2 untuk sesi browser historis, E3 untuk validasi JWT backend, E4 untuk pengujian stack, dan demonstrasi manual M3. E4 mencatat empat tes P3 lulus tanpa error/failure maupun skip, dengan durasi 61,37 detik. Angka tersebut adalah hasil suite yang disimpan; kesimpulan per alur di bawah mengikuti request dan respons yang benar-benar terlihat pada artefak.

4.4.1 Isolasi Kredensial Sumber

E1 merekam penolakan tujuh request tidak sah pada kedua mock. Pada suite E4, `test_01_cross_service_credentials_rejected` menguji kredensial silang dan request tanpa kredensial pada endpoint data `/seismic-events` serta `/volcanic-reports`. Keduanya merupakan dasar pengujian isolasi kredensial.

Artefak terbaru `P3/1-new.png` pada commit `4a1973b` memperlihatkan dua penolakan HTTP 401: bearer PVMBG ditolak BMKG, dan API key BMKG ditolak PVMBG. Respons keduanya menyatakan kredensial instansi yang sesuai diperlukan. Bukti ini menggantikan `P3/1.png` sebagai observasi manual utama dan mendukung isolasi kredensial dua arah. Artefak lama tetap dipertahankan sebagai riwayat.

Kontrol dengan kredensial benar pada gambar terbaru masih menghasilkan 404 untuk BMKG dan 405 untuk PVMBG akibat pemilihan route/method. Karena itu label *Authorized* pada keluaran menu tidak ditafsirkan sebagai keberhasilan membaca data. Keberhasilan akses dengan kredensial benar pada endpoint data tetap merujuk E1/E4; gambar terbaru digunakan untuk dua penolakan kredensial silang yang terlihat.

4.4.2 Otorisasi Ringkasan dan Data Penuh

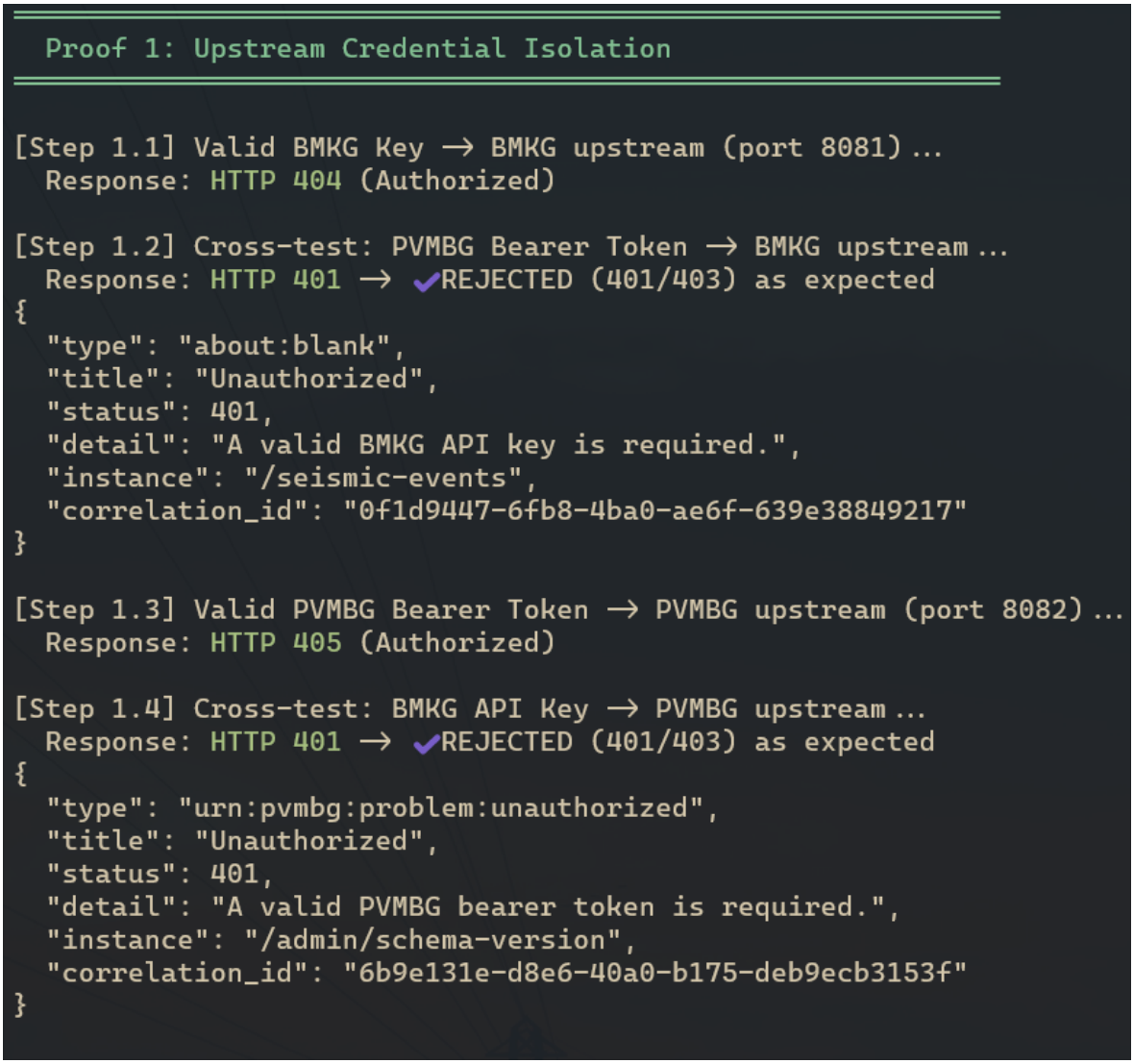
M3 memperlihatkan request akun service ber-role MEDIA ke `/api/v1/hazards` memperoleh 200 dan hanya tujuh field ringkasan. Permintaan eksplisit `view=full` dengan role yang sama memperoleh 403 dari server, beserta correlation ID. Ketika role FIELD_TEAM meminta proyeksi penuh, respons 200 memuat `source_ref_id`, koordinat, dan attributes. Hal ini menunjukkan bahwa pembatasan data berlaku pada API, bukan hanya pada tampilan.

E4 melengkapi tangkapan layar dengan pemeriksaan proyeksi penuh untuk FIELD_TEAM dan BNPB_INTERNAL serta penolakan detail hazard penuh bagi MEDIA. Identitas pada tes tersebut adalah akun service `svc-media`, `svc-lapangan`, dan akun service BNPB. Demonstrasi ini membuktikan enforcement role pada request API; login tiga akun pengguna melalui hosted UI tercatat terpisah pada E2.

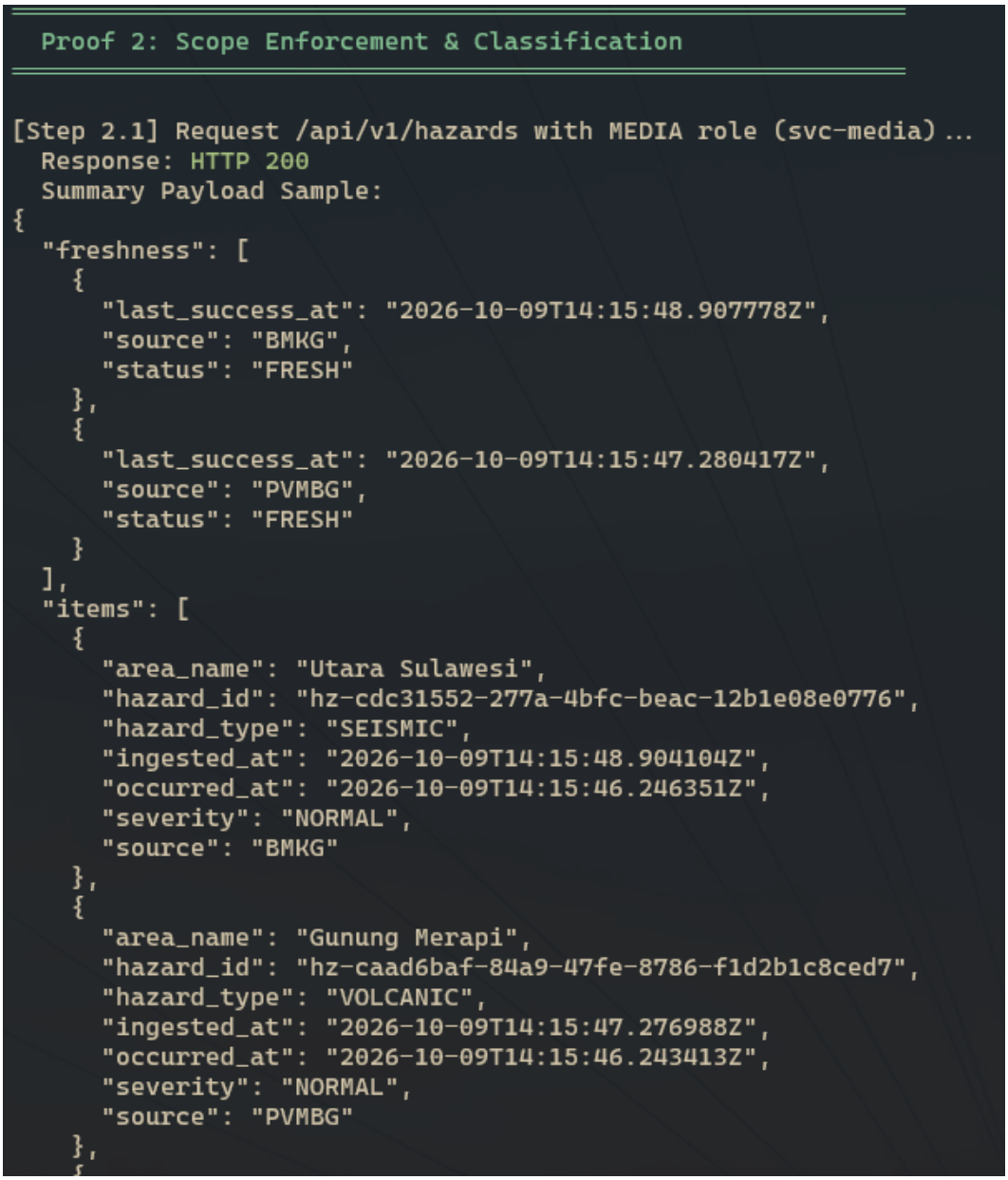
4.4.3 Expiry Token dan Pembaruan Akses

Pada `P3/3-1.png` dan `P3/3-2.png`, token akun service Tim Lapangan mula-mula menghasilkan 200. Setelah menunggu 62 detik, token yang sama menghasilkan 401. Penerbitan token baru menghasilkan akses 200, sedangkan token pertama tetap ditolak dengan 401. Dengan demikian, artefak manual mendukung berlakunya expiry pada jalur gateway serta pemulihan akses menggunakan token baru.

Menu demo dan `test_03_token_ttl_expiry_and_refresh_flow` meminta token pengganti dengan grant `client_credentials`. Ini adalah penerbitan ulang token service; tidak melibatkan refresh token sesi pengguna. Penolakan token pertama setelah expiry juga tidak mengukur kecepatan revokasi token yang masih berlaku.



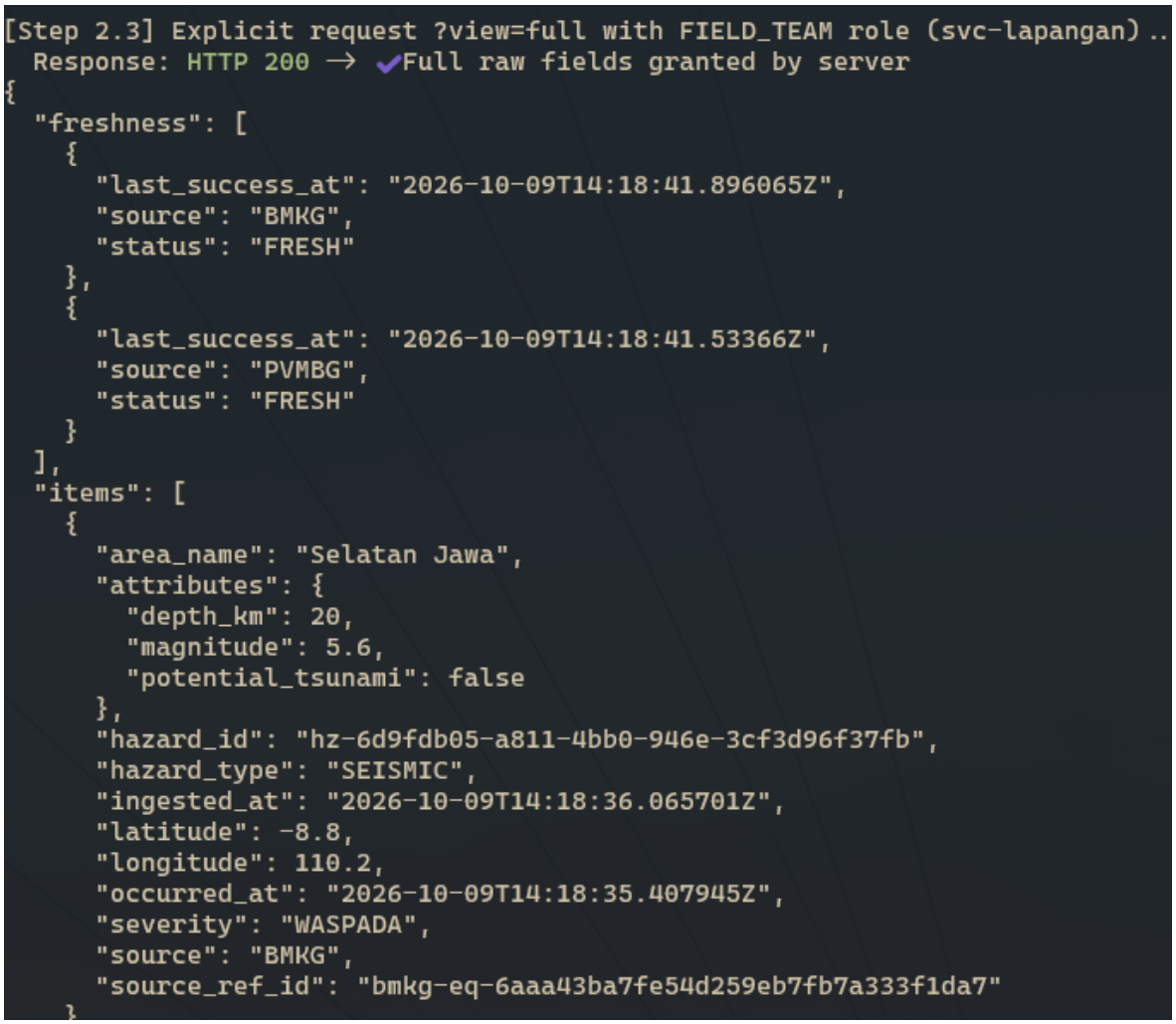
Gambar 4.1: M3 terbaru: kredensial silang ditolak dengan HTTP 401 pada BMKG maupun PVMBG. Status 404/405 pada kontrol terpisah tidak digunakan sebagai bukti keberhasilan akses data.



Gambar 4.2: M3: respons summary untuk role MEDIA; record tidak memuat koordinat, referensi sumber, ataupun attributes.



Gambar 4.3: M3: permintaan eksplisit data penuh oleh MEDIA ditolak server dengan HTTP 403.



Gambar 4.4: M3: role FIELD_TEAM memperoleh HTTP 200 dan field penuh.

Tambahan M3 `P3/3-1-alternative.png` dan `P3/3-2-alternative.png` merekam respons request `refresh` pada browser dengan dashboard aktif. Kedua respons memuat identitas pengguna yang sama, nama BNPB Pusat, role `BNPB_INTERNAL`, serta mode `oidc`. Nilai `expires_at` berubah dari 03:30:33 menjadi 03:31:28 UTC pada 10 Oktober 2026, atau bertambah 55 detik, sementara request polling dashboard berlanjut. Ini mendukung pembaruan sesi browser BNPB Pusat pada stack terbaru selama penggunaan dashboard, sesuai penjadwalan refresh sekitar lima detik sebelum TTL 60 detik berakhir.

Field aman	**Respons refresh pertama**	**Respons refresh berikutnya**
Nama pengguna	BNPB Pusat	BNPB Pusat
Role dan mode	`BNPB_INTERNAL`, `oidc`	`BNPB_INTERNAL`, `oidc`
`expires_at` (UTC)	`2026-10-10T03:30:33.000Z`	`2026-10-10T03:31:28.000Z`

Tabel 4.2: Transkripsi field aman dari dua screenshot refresh M3 terbaru. Identitas tetap sama dan expiry bertambah 55 detik; nilai CSRF tidak disalin.

Kedua screenshot tersebut melengkapi E2, yang mencatat 45 pemeriksaan browser/provider pada 6 Oktober dengan arsitektur bridge saat itu. Cakupan M3 terbaru adalah pembaruan sesi BNPB Pusat; bukti tersebut tidak memperlihatkan sesi pengguna Tim Lapangan dibiarkan kedaluwarsa lalu dipulihkan, ataupun request eksplisit memakai token lama setelah refresh. Kesimpulan kedua skenario itu tetap dibedakan dari refresh browser yang sudah terlihat.

4.4.4 Konfigurasi Secret dan Ringkasan Hasil

M3 `P3/4-1.png` dan `P3/4-2.png` memperlihatkan penelusuran pola key/token/secret pada repository dan tersedianya placeholder konfigurasi. E4 memeriksa variabel yang diwajibkan oleh suite pada `.env.example` serta pola tertentu untuk kredensial literal. Pemeriksaan ini mendukung penggunaan environment pada komponen terkait, dengan cakupan pola terbatas: kredensial pendek atau bentuk YAML tanpa tanda kutip tidak seluruhnya tercakup. Password pengembangan literal pada Compose canonical database tetap menjadi batasan konfigurasi yang telah dijelaskan sebelumnya.

Kriteria	**Bukti tersedia**	**Kesimpulan dan batas cakupan**
Isolasi kredensial mock	E1; E4; M3 terbaru `P3/1-new.png`.	Kredensial silang ditolak 401 pada kedua mock. Kontrol akses data yang berhasil mengacu E1/E4.
Pembatasan role API	E4 tes scope; M3 `P3/2-1.png` sampai `P3/2-3.png`.	MEDIA summary 200/full 403; FIELD_TEAM full 200. E4 juga memeriksa BNPB_INTERNAL.
Expiry dan token pengganti	E4; M3 `P3/3-1.png` dan `P3/3-2.png`.	Token service lama 401 sesudah 62 detik; token baru 200. Alur memakai client credentials.
Login, cookie, refresh browser	E2; M3 `P3/3-1-alternative.png` dan `P3/3-2-alternative.png`.	Sesi browser BNPB Pusat tetap aktif, identitas sama, dan expiry bertambah 55 detik. Expiry sesi Tim Lapangan dan penolakan token lama sesudah refresh tidak ditunjukkan oleh screenshot baru.
Validasi JWT backend	E3: 15 tes utama Auth Aggregator lulus.	Memeriksa issuer, expiry, algoritme, rotasi JWKS, service token, dan provider unavailable dengan server tes.
Konfigurasi tanpa secret literal	E4 dan M3 `P3/4-1.png`, `P3/4-2.png`.	Pemeriksaan pola dan placeholder tersedia; temuan password pengembangan literal membatasi klaim kepatuhan menyeluruh.

Tabel 4.3: Hasil P3 dengan pembedaan alur service, sesi browser, dan konfigurasi.

Nilai token, cookie, password, dan client secret tidak diperlukan dalam laporan. Gambar expiry yang menyertakan fragmen token dan dua screenshot refresh terbaru yang memuat CSRF token disimpan sebagai artefak rujukan asli, tetapi tidak dimuat langsung ke PDF. Uraian dan tabel hanya mencantumkan field aman, urutan request, waktu tunggu, serta status HTTP. Panduan pengujian sesi browser serta E2E tersedia di `docs/auth-testing.md` dan `docs/e2e-testing.md`.

Bab 5

P4: Layanan dan Penyimpanan

5.1 Rancangan Solusi

Pemisahan layanan mengikuti tanggung jawab pengolahan data dan cara penyampaiannya. Aggregator mengintegrasikan sumber eksternal ke model kanonis; Event Live Feed melayani aliran event; Webhook Provider mengirim callback; Danger Notification memilih peringatan untuk push notification. Gateway menerjemahkan kontrak HTTP menjadi panggilan gRPC, sehingga layanan domain tidak perlu mengimplementasikan ulang seluruh antarmuka publik.

Consumer telah diimplementasikan dalam tiga bahasa: Rust untuk Event Live Feed, Go untuk Webhook Provider, dan C# untuk Danger Notification. Kontrak Protobuf menjadi batas pertukaran data lintas bahasa. Compose membangun masing-masing image dari direktori layanannya dan menghubungkannya melalui jaringan Docker. Pemisahan proses memungkinkan restart satu consumer tanpa harus mematikan consumer lain, meskipun kemandirian operasional tetap dibatasi dependency yang dipakai bersama.

Tabel 5.1: Pembagian tanggung jawab dan penyimpanan layanan

Layanan	**Teknologi**	**Tanggung jawab dan data**
Aggregator	Go, gRPC 9091	Normalisasi BMKG/PVMBG; hazard kanonis, watermark, dan outbox pada PostgreSQL.
Event Live Feed	Rust, gRPC 9093	Subscription event, filter/proyeksi, cursor, dan replay terbatas dalam memori proses.
Webhook Provider	Go, gRPC 9094	Subscription webhook, delivery log, serta deduplikasi pengiriman pada SQLite.
Danger Notification	C#, gRPC 9095	Filter SIAGA/AWAS; snapshot notifikasi, revisi, dan outbox push pada SQLite; pengiriman melalui FCM.
API Gateway	Go, HTTP	Validasi kontrak, pemetaan HTTP–gRPC, dan SSE; tidak menjadi pemilik data hazard.

Aggregator memiliki tabel `hazard_events` dengan kunci `hazard_id` dan batas unik pasangan `source` serta `source_ref_id`. Atribut umum disimpan sebagai kolom bertipe, sedangkan atribut yang berbeda menurut sumber disimpan dalam JSONB. Kombinasi ini memungkinkan filter lintas sumber tanpa membuang informasi khusus sumber. Tabel `ingest_watermarks` menyimpan kemajuan ingestion dan kesegaran, `outbox_events` menyimpan publication yang belum dikirim, dan `outbox_dead_letters` menyimpan publication yang mencapai batas gagal. Schema yang diterapkan saat startup berada pada `services/aggregator/internal/store/postgres.go`.

Kepemilikan logis Canonical Store berada pada Aggregator. Danger Notification menerima snapshot hazard dari event broker dan menyimpannya pada SQLite melalui `EfNotificationStore`. Daftar dan detail notifikasi dibaca dari penyimpanan lokal ini, sedangkan status kesegaran sumber tetap diperoleh melalui adapter gRPC Aggregator. FCM mengelola subscription token ke topic. Dengan demikian, data notifikasi tidak memerlukan query langsung ke tabel hazard milik Aggregator.

Tabel `notifications` memakai ID publication sebagai primary key dan menyimpan snapshot Protobuf hazard, `hazard_id`, waktu publication, penanda `IsCurrent`, status push, jumlah kegagalan, jadwal retry, dan lease pekerjaan. Revisi baru menandai revisi lama hazard yang sama sebagai tidak current; API daftar/detail menampilkan revisi current. Upsert dan pembaruan revisi menggunakan transaksi SQLite. `PushOutboxWorker` mengambil pekerjaan yang waktunya sudah tiba, mengirim melalui FCM, lalu mencatat hasilnya. Compose memasang `notification_data` pada `/data`, dengan database `notifications.db`. Registrasi layanan dan migrasi berada pada `Program.cs` serta adapter `Adapters/Outbound/Persistence/` di layanan Danger Notification.

Webhook Provider memiliki database SQLite tersendiri untuk `webhook_subscriptions`, `webhook_deliveries`, dan `delivery_dedup`. Mode WAL digunakan dan pool dibatasi satu koneksi terbuka. Kunci gabungan `event_id` dan `webhook_id` pada tabel dedup menandai delivery yang berhasil. Implementasi berada pada `services/webhook-provider/internal/store/sqlite.go`. Compose memasang volume `webhook_data` pada `/data`; DSN contoh mengarahkan database ke `file:/data/webhook.db`.

Event Live Feed menggunakan abstraksi `ReplayStore` dengan implementasi `MemoryStore`. Buffer menyimpan urutan event dan himpunan ID yang sudah diterima, dilindungi mutex, lalu menyebarkan event ke kanal broadcast. Default Compose membatasi replay pada 600 detik atau 10.000 entri, sedangkan kanal live memiliki kapasitas 256 entri. Ketika subscriber tertinggal, backlog dapat dibaca kembali selama belum keluar dari jendela replay. Restart membentuk epoch baru dan cursor lama ditolak, sehingga klien mengetahui bahwa riwayat sebelumnya tidak lagi tersedia. Sumbernya adalah `services/event-live-feed/src/core/ports.rs` dan `services/event-live-feed/src/adapters/outbound/memory_store.rs`.

5.2 Justifikasi dan Alternatif

Pemilihan penyimpanan mengikuti kebutuhan masing-masing layanan. PostgreSQL memberikan transaksi untuk menyimpan hazard dan outbox bersama, keunikan identitas sumber, serta indeks untuk penyaringan dan paginasi. SQLite memadai untuk volume subscription dan log webhook pada satu instance, dengan biaya operasi rendah. Memori proses menyediakan replay live-feed yang ringan ketika kontraknya memang membatasi umur riwayat.

Alternatif satu database bersama untuk semua layanan mengurangi jumlah komponen, tetapi memungkinkan perubahan schema satu layanan memengaruhi layanan lain secara langsung. Rancangan yang dipilih membatasi akses melalui kontrak gRPC atau event. Fleksibilitas penyimpanan berarti implementasi penyimpanan dapat disesuaikan per kebutuhan; bukan berarti mengganti database tanpa migrasi data, perubahan adapter, dan pengujian ulang.

Untuk feed, penyimpanan eksternal seperti Redis atau PostgreSQL dapat memperpanjang replay lintas restart. Implementasi saat ini memilih memori, sehingga biaya serialisasi dan I/O penyimpanan lebih kecil, tetapi tidak mengklaim riwayat permanen. Untuk webhook berskala lebih besar, PostgreSQL dapat menggantikan SQLite agar beberapa worker mempunyai state dedup bersama, dengan tambahan koordinasi pekerjaan dan pengendalian duplikasi.

Tabel 5.2: Alternatif penyimpanan dan pertimbangannya

Pilihan	Kelebihan untuk kasus ini	Biaya atau batasan
PostgreSQL, dipakai Aggregator	Transaksi hazard–outbox, constraint identitas, dan kolom JSONB untuk atribut sumber.	Memerlukan proses database dan pengelolaan koneksi; kapasitasnya dipakai seluruh pembacaan kanonis.
SQLite, dipakai webhook	Penyimpanan lokal persisten tanpa server database tambahan; schema dan dedup mudah diperiksa.	Satu penulis dan satu volume lokal membatasi replikasi horizontal.
MongoDB, alternatif	Dokumen fleksibel untuk payload sumber yang beragam.	Perlu merancang kembali indeks gabungan, transaksi publication, dan kontrak query; fleksibilitas payload sudah dapat ditampung JSONB.
Redis, alternatif replay	Data replay dan dedup dapat ditempatkan di luar proses feed untuk dipakai beberapa instance.	Menambah dependency; TTL, eviction, persistensi, dan posisi cursor perlu dirancang eksplisit.

5.3 Batasan Solusi

Pemisahan image dan proses belum membuktikan bahwa setiap layanan dapat direplikasi tanpa perubahan. Publisher outbox belum memakai mekanisme claim/lock antar-worker pada query pengambilan record. Feed memiliki replay dan epoch lokal per proses. SQLite webhook membatasi penulisan serta memerlukan strategi berbagi state sebelum menjalankan beberapa replica. Keterbatasan ini perlu diselesaikan sebelum menyatakan dukungan penskalaan horizontal yang transparan.

Durability harus dibedakan per komponen. PostgreSQL menggunakan volume `aggregator_data`; SQLite menggunakan `webhook_data` dan `notification_data` untuk masing-masing layanan. Data tersebut tetap bergantung pada lifecycle volume dan prosedur backup. Feed tidak persisten setelah restart. Notification menyimpan status push dan identitas publication secara lokal; status diterima FCM belum merupakan bukti pesan telah tampil pada perangkat pengguna. RabbitMQ mendeklarasikan queue durable, tetapi `compose.yaml` belum memasang volume data broker secara eksplisit. Karena itu, ketahanan seluruh state setelah container dibuat ulang belum menjadi jaminan sistem.

Penyimpanan hazard mempertahankan satu identitas kanonis per referensi sumber, tetapi upsert tidak membandingkan seluruh isi untuk memastikan perubahan substantif. Publication baru tetap dapat dibentuk untuk data yang diambil ulang. Pemisahan database dan penggunaan constraint tidak dengan sendirinya menyediakan pemrosesan event tepat satu kali.

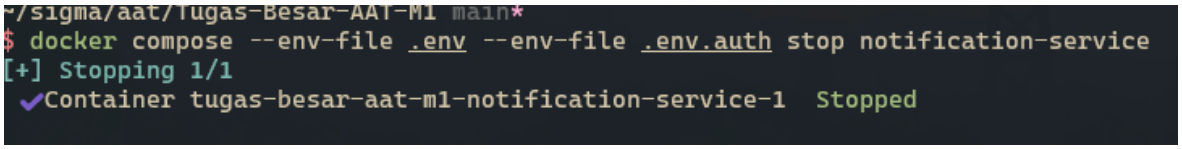
5.4 Proof-of-Concept dan Keterlacakan

Bukti manual M4 pada `docs/tex/evidence/P4/` mencakup rangkaian awal dan tambahan `1-3-new.png` pada commit `4a1973b`. Ringkasan eksekusi otomatis E4 pada `docs/tex/evidence/problem_set_report.md`, dibuat 10 Oktober 2026, mencatat tiga tes P4 lulus, tanpa failure maupun skipped. Cakupan kedua jenis bukti dijelaskan berikut; label lulus tes tidak memperluas observasi melampaui assertion dan screenshot yang tersedia.

5.4.1 Pemisahan Proses dan Pengoperasian Layanan

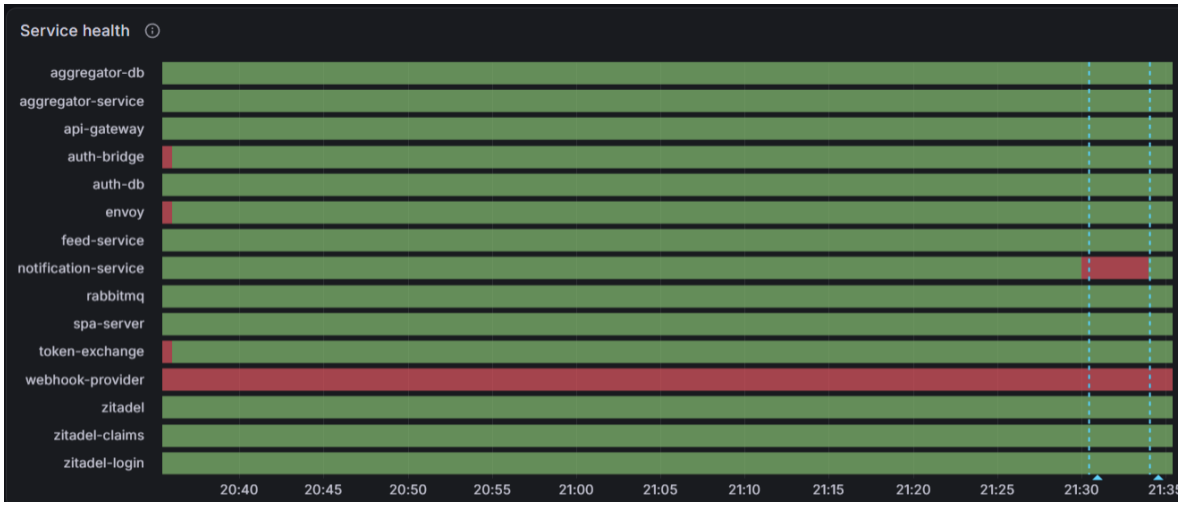
Pada M4, operator menghentikan `notification-service`. Halaman notifikasi kemudian menampilkan respons 503 untuk endpoint yang bergantung pada layanan tersebut, sementara

feed tetap aktif dan status sumber tetap tersedia. Setelah layanan dibangun dan dijalankan kembali, halaman notifikasi kembali menampilkan data. Urutan ini direkam pada `P4/1-1.png` sampai `P4/1-4.png`.



Gambar 5.1: M4: penghentian container Danger Notification secara terpisah.

Grafik health memperlihatkan interval notification tidak tersedia, sedangkan Aggregator, feed, gateway, dan database tetap terpantau aktif. Webhook memang sedang tidak aktif pada interval grafik tersebut; gambar ini tidak digunakan untuk menyatakan semua layanan hidup bersamaan.



Gambar 5.2: M4: observasi health ketika notification dihentikan dan dipulihkan.

Tes otomatis `tests/test_problem_4.py` melengkapi observasi dengan menghentikan satu layanan, meminta hazard BMKG melalui API, memeriksa respons 200 beserta data, lalu menjalankan kembali layanan. Ini mendukung ketersediaan jalur baca independen selama penghentian yang diuji. Bukti tambahan `P4/1-3-new.png` memakai perintah berikut:

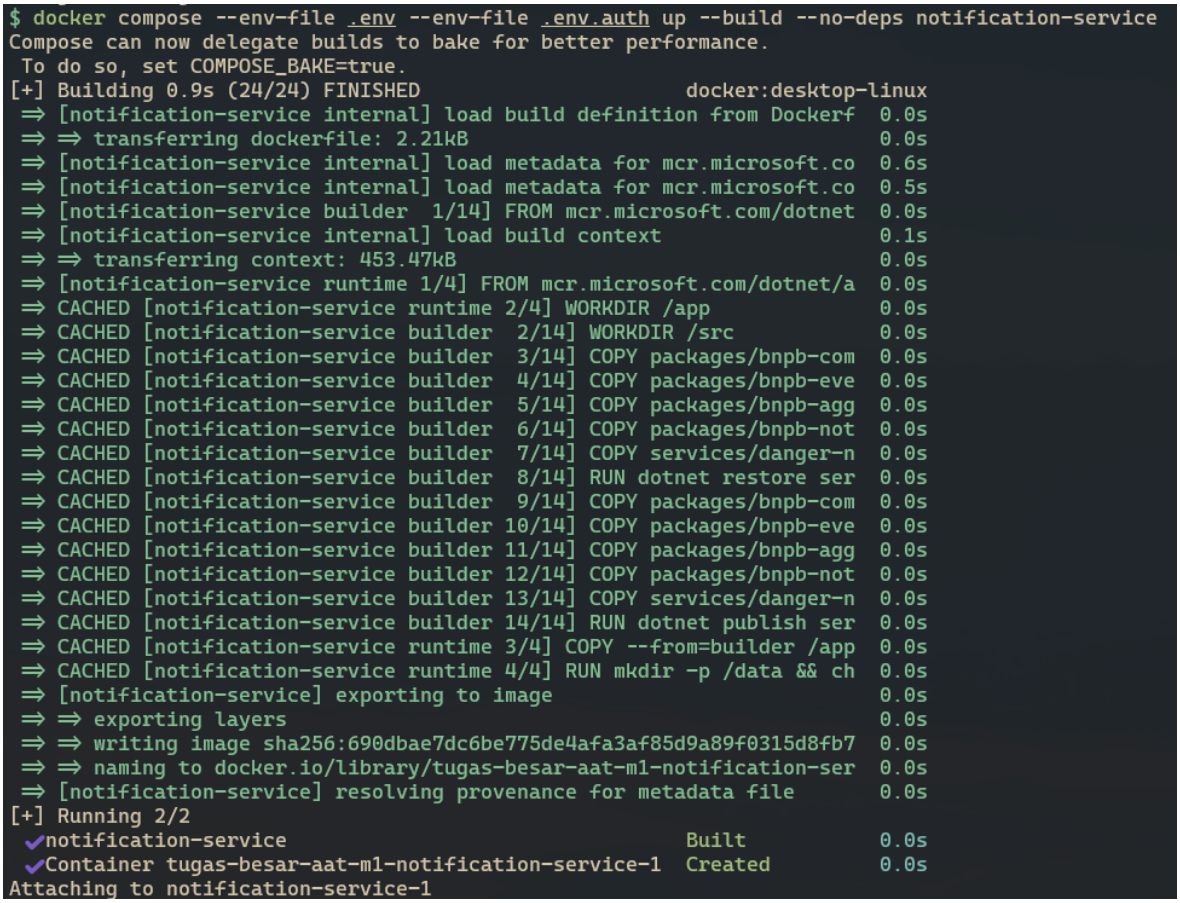
```
docker compose --env-file .env --env-file .env.auth up --build --no-deps notification-service
```

Output memperlihatkan 24 langkah build hanya untuk Notification, image berstatus Built, dan container Notification berstatus Created lalu di-attach. Sebagian besar langkah memakai build cache. Opsi `--no-deps` membatasi operasi pada target sehingga dependency tidak ikut dibangun atau dijalankan oleh perintah ini. Bukti ini menggantikan `P4/1-3.png` sebagai rujukan build satu layanan; screenshot lama tetap tersimpan sebagai riwayat.

Grafik health dan pembacaan API pada rangkaian awal melengkapi observasi ketersediaan jalur independen. Screenshot build baru tidak memuat perbandingan ID/start time seluruh container atau health setelah attach, sehingga tidak digunakan sebagai catatan lengkap riwayat restart setiap container di luar interval yang direkam.

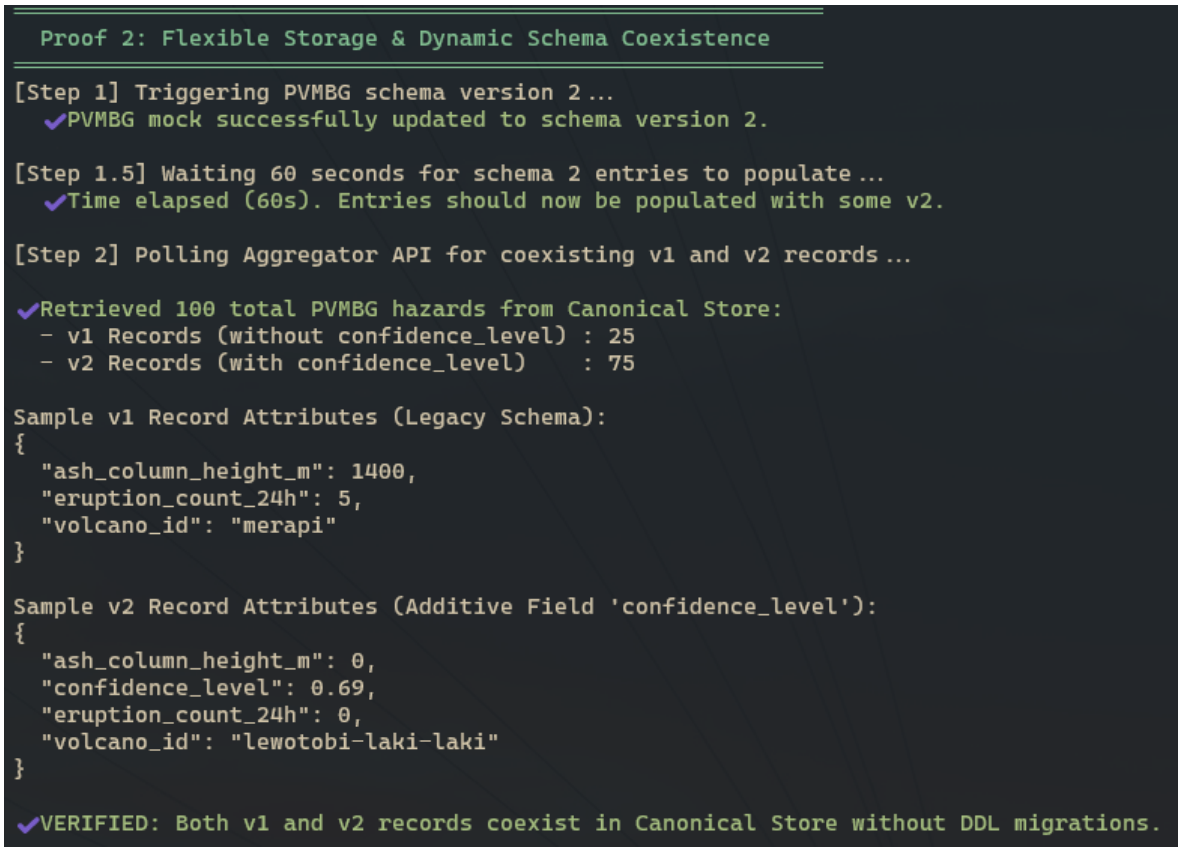
5.4.2 Atribut Dinamis dan Koeksistensi Skema

Setelah kontrol PVMBG diubah ke versi 2 dan ditunggu 60 detik, pembacaan 100 hazard PVMBG melalui Canonical Store menemukan 25 record tanpa `confidence_level` dan 75



Gambar 5.3: M4 terbaru: build dan pembuatan container Notification dengan `--no-deps`; output hanya menampilkan layanan target.

record dengan atribut tersebut. Screenshot menampilkan contoh kedua bentuk atribut, termasuk nilai confidence 0,69. Pemeriksaan tabel memperlihatkan bahwa `attributes` bertipe JSONB; `confidence_level` tidak menjadi kolom terpisah.



Gambar 5.4: M4: record PVMBG sebelum dan sesudah penambahan atribut tersedia bersamaan melalui API.

Hasil ini mendukung rancangan penyimpanan fleksibel dan koeksistensi data v1/v2. Tes otomatis memeriksa keberadaan kedua kelompok record serta tipe numerik confidence. Screenshot DDL yang tersedia adalah satu snapshot struktur, bukan pasangan hasil sebelum/sesudah. Oleh sebab itu, tidak ada angka perubahan DDL atau hasil perbandingan hash schema yang diklaim.

5.4.3 Kepemilikan Data dan Batas Pengujian

M4 `P4/3.png` memperlihatkan inspeksi konfigurasi serta keberhasilan pembacaan status sumber melalui API. Pada kode aplikasi, operasi Canonical Store ditangani Aggregator; consumer memperoleh data lewat event atau API. Pemeriksaan otomatis mencari variabel `CANONICAL_STORE_DSN` secara literal dalam blok Compose layanan lain. Pemeriksaan ini menunjukkan konvensi konfigurasi yang diperiksa, tetapi tidak membuktikan pembatasan akses database pada tingkat jaringan atau hak pengguna.

Konfigurasi saat ini masih memuat `env_file:.env` pada Webhook Provider, sehingga variabel Canonical Store dalam berkas tersebut ikut masuk ke lingkungan container. Database juga berada pada jaringan bersama dan port PostgreSQL diekspos ke host; collector observabilitas mempunyai koneksi PostgreSQL untuk metrik. Dengan demikian, istilah kepemilikan pada laporan ini berarti pemisahan tanggung jawab aplikasi. Pembatasan credential per layanan, role metrik khusus, dan jaringan database perlu diterapkan bila kriteria akses eksklusif hendak dijamin secara teknis.

```
bnpb_canonical=# \d hazard_events
                        Table "public.hazard_events"
    Column     |           Type           | Collation | Nullable |   Default
---------------+--------------------------+-----------+----------+-------------
 hazard_id     | character varying(128)   |           | not null |
 source        | character varying(32)    |           | not null |
 source_ref_id | character varying(128)   |           | not null |
 hazard_type   | character varying(32)    |           | not null |
 severity      | character varying(32)    |           | not null |
 area_name     | character varying(255)   |           | not null |
 latitude      | double precision         |           | not null |
 longitude     | double precision         |           | not null |
 occurred_at   | timestamp with time zone |           | not null |
 ingested_at   | timestamp with time zone |           | not null |
 attributes    | jsonb                    |           | not null | '{}'::jsonb
Indexes:
    "hazard_events_pkey" PRIMARY KEY, btree (hazard_id)
    "idx_hazard_pagination" btree (occurred_at DESC, hazard_id)
    "idx_hazard_severity" btree (severity)
    "idx_hazard_source" btree (source)
    "idx_hazard_type" btree (hazard_type)
    "uq_source_ref" UNIQUE CONSTRAINT, btree (source, source_ref_id)
```

Gambar 5.5: M4: struktur tabel hazard menggunakan JSONB untuk atribut sumber.

Tabel 5.3: Cakupan bukti P4 dan batas interpretasi

Objek	**Hasil dan batas bukti**
Independensi proses	Stop dan pemulihan diamati; bukti baru memperlihatkan build target dengan `--no-deps`. Health jalur lain tersedia pada rangkaian awal. Identitas semua container belum dibandingkan.
Atribut dinamis	25 record tanpa confidence dan 75 dengan confidence terbaca; kolom JSONB diperlihatkan.
Kepemilikan Canonical Store	Jalur aplikasi melalui Aggregator/event ditunjukkan; konfigurasi belum menjamin isolasi credential dan jaringan.
Persistensi lintas restart	Volume PostgreSQL, webhook, dan notification tersedia pada kode. Bukti M4 tidak membandingkan ID/jumlah data ketiganya sebelum dan sesudah restart.

Bukti historis E1 tetap hanya berlaku untuk persistensi database mock yang ketika itu diuji; tidak digunakan sebagai pengganti pengujian volume Canonical Store, webhook, atau notification. Pengujian unit Aggregator E3 mendukung transformasi atribut, sedangkan E4 dan M4 menambahkan pengamatan pada stack berjalan. Sumber bukti terdaftar pada `docs/tex/evidence/manifest.json`; petunjuk pengujian tersedia pada `docs/e2e-testing.md`.

Bab 6

P5: Distribusi Event

6.1 Rancangan Solusi

Aggregator memublikasikan perubahan hazard melalui exchange RabbitMQ bertipe fanout. Setiap consumer memiliki queue sendiri, sehingga satu publication disalin ke Event Live Feed, Webhook Provider, dan Danger Notification. Ketiga layanan dapat memproses publication yang sama untuk tujuan berbeda. Menempatkan ketiganya pada satu queue akan menghasilkan pembagian pekerjaan antar-consumer, bukan penyalinan event ke seluruh layanan.

Tabel 6.1: Topologi distribusi event

Komponen	**Nama contoh/default**	**Peran**
Exchange	`bnpb.hazards.v1`	Fanout durable.
Queue feed	`bnpb.event-live-feed.v1`	Event streaming dan replay.
Queue webhook	`bnpb.webhook-provider.v1`	Callback subscription.
Queue notification	`bnpb.danger-notification.v1`	Push SIAGA/AWAS.
DLQ per consumer	Nama layanan dengan akhiran `.dlq.v1`	Menampung pesan yang ditolak tanpa requeue.

Nama exchange dan queue berasal dari konfigurasi. Producer hanya mendeklarasikan exchange; tiap consumer mendeklarasikan queue durable, dead-letter queue, dan binding miliknya. Routing dead-letter menggunakan default exchange dengan routing key nama DLQ. Ketiga consumer menggunakan prefetch satu pesan dan acknowledgement manual. Pengaturan ini membatasi pesan yang belum diakui, tetapi juga membuat pekerjaan lambat dalam satu consumer menahan pekerjaan berikutnya pada consumer tersebut.

Publication memakai envelope Protobuf `HazardEventPublished`, berisi `event_id`, `schema_version`, sumber, publisher, correlation ID, waktu publication, dan payload hazard kanonis. Implementasi saat ini menggunakan versi schema 1 dan publisher `aggregator`. `event_id` berupa UUID publication dan berbeda dari `hazard_id` yang stabil untuk satu referensi sumber. Pembentukan envelope dapat ditelusuri pada `services/aggregator/internal/ingest/mapper.go`.

Alur penyimpanan dan publication menggunakan pola transactional outbox:

1. Poller memetakan data sumber ke hazard kanonis, lalu membuka transaksi PostgreSQL.
2. Upsert hazard dan penyisipan envelope ke `outbox_events` menggunakan transaksi yang sama, kemudian transaksi di-commit.

3. Publisher mengambil paling banyak 20 publication pending setiap 200 ms dan mengirim pesan persisten ke exchange.
4. Publisher menunggu publisher confirm dari broker sebelum mengisi `published_at`. Kegagalan publish menaikkan retry record.
5. Pada batas retry, contoh konfigurasi lima percobaan, record ditandai gagal dan disalin ke tabel `outbox_dead_letters`.

Implementasi publisher terdapat pada `services/aggregator/internal/broker/publisher.go` dan `services/aggregator/internal/store/outbox_store.go`. Tabel dead-letter outbox merupakan state PostgreSQL producer, berbeda dari DLQ RabbitMQ masing-masing consumer. Kegagalan koneksi awal broker diulang setiap dua detik; retry koneksi tersebut bukan penghitung percobaan publish setiap record.

Setiap consumer menentukan titik acknowledgement sesuai pekerjaannya:

- **Event Live Feed** memvalidasi envelope, menambahkan event ke replay buffer, lalu ack. ID yang masih berada dalam replay window dideduplikasi. Envelope rusak atau tidak didukung langsung di-nack tanpa requeue. Koneksi diulang dengan backoff 1–30 detik.
- **Webhook Provider** mencari subscription yang sesuai filter, memeriksa dedup, membentuk payload sesuai view, dan mengirim HTTP POST dengan tanda tangan HMAC-SHA256 pada `X-BNPB-Signature`. Hasil dicatat ke SQLite dan delivery sukses ditandai untuk dedup. Default dispatcher melakukan lima percobaan dengan timeout 10 detik per request serta jeda 1, 2, 4, dan 8 detik.
- **Danger Notification** memproses envelope dan memilih severity SIAGA/AWAS. Event yang memenuhi filter disimpan secara transaksional pada SQLite sebagai notifikasi dan pekerjaan push pending, kemudian pesan broker di-ack. Event di luar filter di-ack tanpa pekerjaan push. `PushOutboxWorker` mengirim melalui FCM secara terpisah dan menyimpan hasil pengiriman beserta jadwal retry. Parse error Protobuf langsung ditolak; error pemrosesan consumer lainnya memperoleh satu redelivery sebelum ditolak menuju DLQ. Ack broker tidak menunggu keberhasilan FCM atau penerimaan pada perangkat.

Kode consumer berada pada `services/event-live-feed/src/adapters/inbound/rabbitmq.rs`, `services/webhook-provider/internal/consumer/consumer.go`, dan `services/danger-notification/BNPB.Notification.Server/Adapters/Inbound/RabbitMqConsumer.cs`. Header trace dibawa melalui AMQP untuk menghubungkan span publish dan consume. Identitas publication membantu menelusuri satu event di beberapa consumer, sementara correlation ID menghubungkannya dengan putaran ingestion.

6.2 Justifikasi dan Alternatif

Fanout dipilih karena semua consumer menerima event hazard yang sama lalu melakukan filter atau proyeksi sesuai fungsi layanan. Producer tidak perlu mengetahui alamat callback maupun perangkat pengguna. Queue terpisah memungkinkan consumer yang berhenti menimbun backlog tanpa memaksa consumer lain berhenti memproses queue masing-masing, selama broker dan resource bersama masih tersedia.

Alternatif panggilan HTTP langsung dari Aggregator ke setiap consumer mengurangi kebutuhan broker, tetapi menjadikan publisher bertanggung jawab atas timeout, retry, dan availability setiap tujuan. Alternatif exchange topic dapat mengurangi lalu lintas dengan routing menurut sumber atau jenis hazard; konsekuensinya ialah penambahan aturan routing dan kesepakatan key. Fanout mempertahankan topologi yang sederhana untuk tiga consumer saat ini.

Alternatif log terdistribusi berbasis offset memberi retensi dan replay yang lebih panjang, tetapi

memerlukan pengelolaan partition, offset, dan consumer group. Kebutuhan milestone dapat ditangani RabbitMQ dan outbox PostgreSQL. Keputusan ini tidak menyamakan queue dengan arsip event permanen: event yang telah di-ack tidak lagi menjadi backlog broker, dan replay feed memiliki retensi sendiri.

6.3 Batasan Solusi

Pada transport broker, persistent message, publisher confirm, manual ack, dan redelivery mendukung pola *at-least-once* selama queue sudah terikat, state broker masih tersedia, dan pesan belum dihentikan oleh kebijakan retry atau DLQ. Confirm producer dan ack consumer mengonfirmasi tahap berbeda [7]. Jaminan tersebut tidak sama dengan keberhasilan efek bisnis untuk setiap subscriber. Semantik sistem belum exactly-once. Publisher confirm dan penandaan `published_at` merupakan dua operasi berbeda, sehingga crash di antaranya dapat menyebabkan publication dikirim ulang. Demikian pula, keberhasilan HTTP/FCM dan acknowledgement broker tidak berada dalam transaksi yang sama. Mekanisme dedup harus dinilai per consumer:

1. Feed hanya mendeduplikasi sepanjang replay window dalam memori. Ack setelah append tidak berarti data tersimpan di disk; restart menghapus replay dan himpunan dedup.
2. Webhook menyimpan dedup persisten per pasangan publication dan subscription, tetapi crash setelah penerima menerima POST dan sebelum penandaan dedup masih dapat menghasilkan pengiriman ulang. Penerima juga perlu menangani duplikasi.
3. Notification memakai ID publication sebagai primary key SQLite. Pengiriman ulang ID yang sama tidak membuat notifikasi baru, dan pembaruan status mempertahankan hasil push yang sudah delivered. Revisi baru suatu hazard mempunyai ID publication berbeda dan menandai revisi sebelumnya sebagai tidak current. Lease outbox membantu koordinasi pengiriman, tetapi crash setelah FCM menerima pesan dan sebelum hasil tersimpan tetap dapat menghasilkan push berulang. Dedup penyimpanan tidak sama dengan exactly-once pada perangkat pengguna.

Retry webhook memiliki keterbatasan khusus: setelah seluruh percobaan HTTP gagal, consumer mengatur kegagalan pemrosesan dan melakukan nack dengan requeue. Belum ada batas total redelivery pada jalur tersebut. Dengan demikian, lima kegagalan dispatcher tidak otomatis memindahkan event ke DLQ. Komentar kode yang menyebut dispatch failure tidak menghalangi ack tidak sesuai dengan alur kondisi aktual; uraian laporan mengikuti alur yang dieksekusi.

Pola outbox juga belum menutup semua jalur kegagalan aplikasi. Poller mengabaikan error pembentukan envelope dan hasil pemanggilan `EnqueueOutbox`. Transaksi SQL yang sama mendukung konsistensi ketika operasinya berhasil, tetapi belum cukup untuk menjanjikan bahwa setiap hazard pasti mempunyai publication pada seluruh jalur error. Pembaruan watermark dan publikasi berulang perlu diuji secara terpisah.

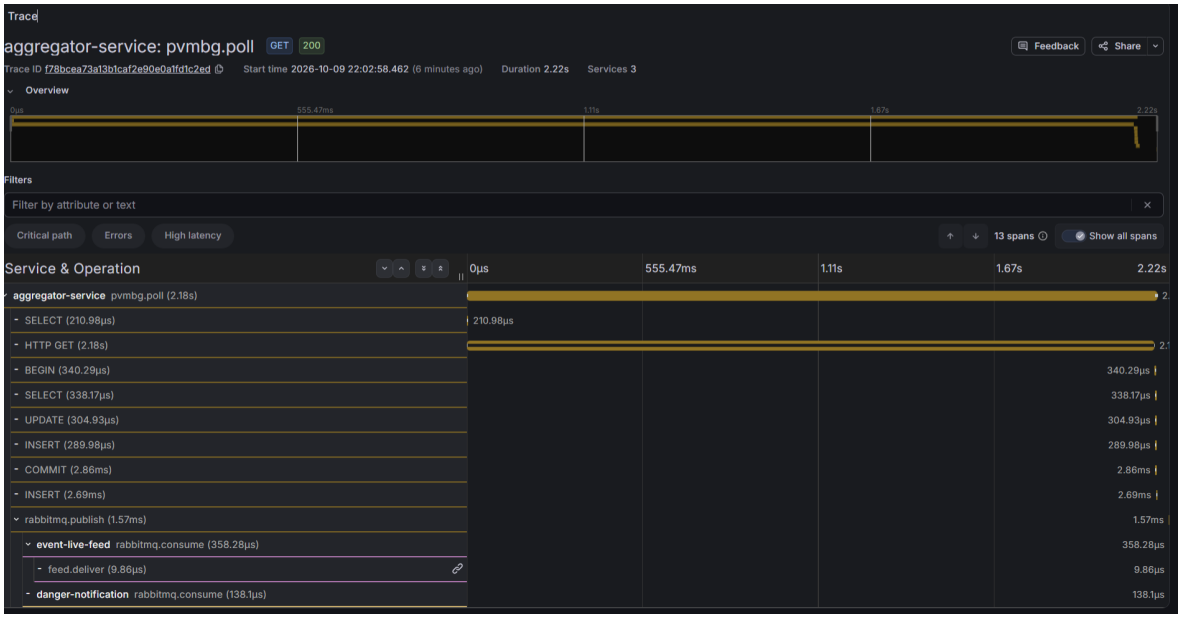
Publication memakai `mandatory=false`; apabila queue consumer belum pernah dibuat dan diikat ketika pesan dipublikasikan, publisher confirm tidak membuktikan consumer tersebut memperoleh salinannya. Compose belum memasang volume broker secara eksplisit. Karena itu, label durable pada queue dan persistent pada pesan tidak cukup untuk menjanjikan pemulihan setelah container broker dibuat ulang. Pemeriksaan schema juga tidak seragam: feed memvalidasi publisher, ID, sumber, dan payload, sedangkan notification terutama bergantung pada parsing serta akses field yang dipakai.

6.4 Proof-of-Concept dan Keterlacakan

Bukti manual M5 pada `docs/tex/evidence/P5/` merekam distribusi event, penghentian consumer, pemulihan backlog, dan pengoperasian consumer tambahan tanggal 9 Oktober 2026. Ringkasan otomatis E4 mencatat empat tes P5 lulus tanpa failure atau skipped pada 10 Oktober 2026. Uraian berikut mengutamakan kondisi yang benar-benar terlihat pada artefak; screenshot beberapa sesi tidak diperlakukan sebagai satu rangkaian waktu yang utuh.

6.4.1 Publication dan Dua Consumer Independen

Trace pada M5 memperlihatkan polling PVMBG oleh Aggregator, transaksi penyimpanan, span `rabbitmq.publish`, serta span `rabbitmq.consume` pada Event Live Feed dan Danger Notification. Keterhubungan span dalam trace yang sama mendukung distribusi satu publication ke dua consumer, tanpa panggilan langsung producer ke endpoint masing-masing.



Gambar 6.1: M5: trace publication Aggregator dengan cabang konsumsi pada Feed dan Notification.

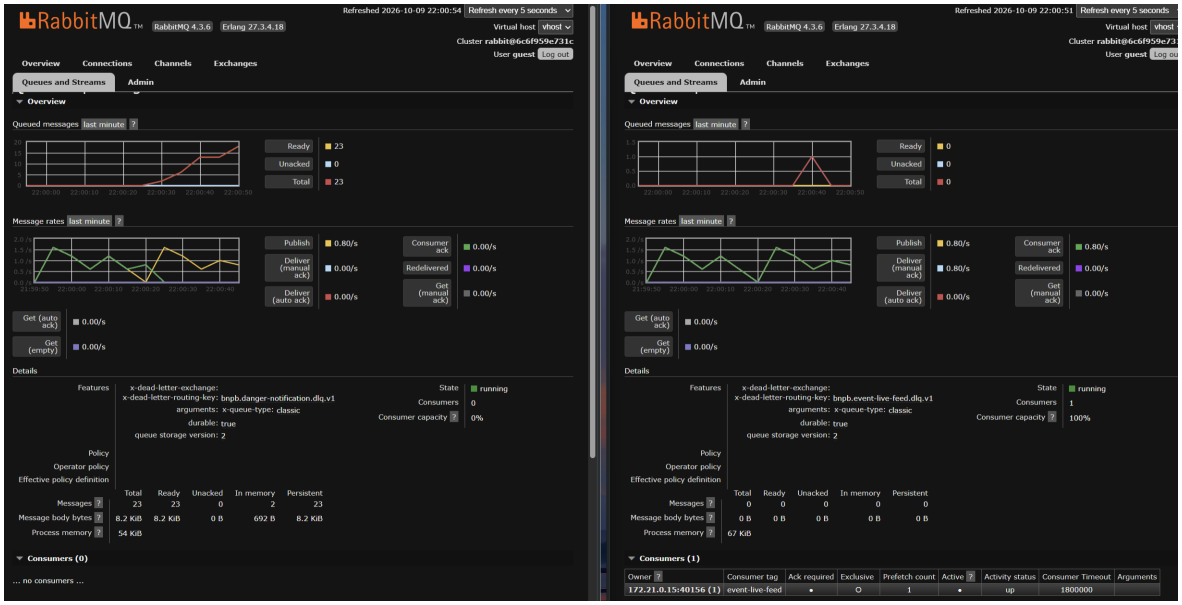
Screenshot `P5/2-1.png` menunjukkan exchange `bnpb.hazards.v1` bertipe fanout dan binding queue Feed serta Notification. Pada sesi tersebut Webhook Provider belum aktif. `P5/2-2.png` dan `P5/2-3.png` memperlihatkan dua queue utama beserta DLQ, consumer aktif, dan laju delivery/ack pada masing-masing queue. Bukti ini memenuhi demonstrasi minimal dua consumer; tidak mengklaim ketiga layanan telah memproses satu ID publication yang terlihat lengkap pada screenshot yang sama.

Tes otomatis membuka SSE dan menerima `hazard.upsert`, lalu memeriksa keberadaan queue Notification. Ringkasan E4 menyebut Feed dan Webhook, sedangkan implementasi tes `test_02` menggunakan Notification. Pasangan yang dipakai pada pembahasan hasil ini adalah Feed–Notification, sesuai kode dan trace M5. Assertion queue tes tersebut belum mencocokkan ID publication; trace manual menjadi bukti tambahan untuk hubungan publish–consume.

6.4.2 Consumer Berhenti dan Pemulihan Backlog

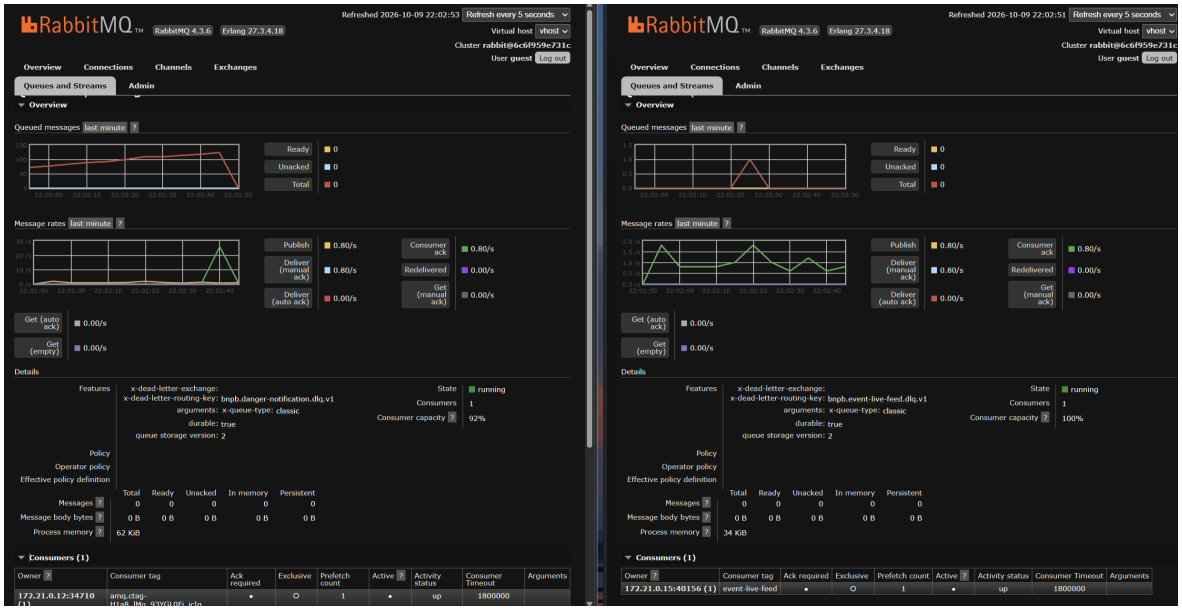
Ketika Notification dihentikan, queue miliknya menunjukkan nol consumer dan 23 pesan ready, sementara queue Feed tetap mempunyai satu consumer serta laju delivery dan ack 0,80 pe-

san/detik. Broker masih menerima publication untuk kedua queue. Kondisi ini menunjukkan bahwa tidak aktifnya satu consumer tidak menghentikan distribusi ke consumer lainnya.



Gambar 6.2: M5: backlog Notification bertambah saat consumernya berhenti, sedangkan Feed tetap memproses event.

Setelah Notification kembali aktif, grafik queue memperlihatkan backlog yang sebelumnya mencapai lebih dari 100 pesan turun menjadi nol. Jumlah consumer kembali satu dan terdapat lonjakan delivery/ack saat pengurasan. Feed tetap aktif selama interval yang diperlihatkan.



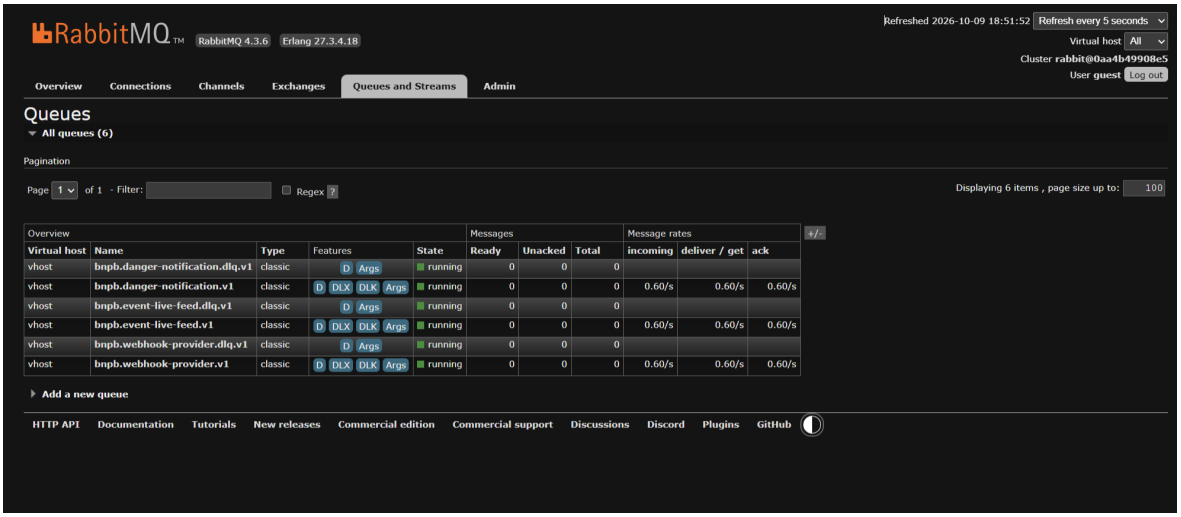
Gambar 6.3: M5: backlog Notification terkuras setelah consumer kembali aktif.

Event yang terbit ketika consumer berhenti tersimpan sebagai pesan ready pada queue yang sudah terikat. Ketika consumer kembali, pesan tersebut baru dikirim dan di-ack; mekanisme ini berbeda dari redelivery pesan yang sudah pernah dikirim tetapi belum di-ack. Pada screenshot laju redelivery bernilai nol, sehingga hasil tersebut tidak digunakan untuk mengklaim pengujian redelivery atau DLQ. Berkurangnya backlog membuktikan pemrosesan broker, bukan jumlah push FCM yang berhasil tampil pada perangkat.

Tes otomatis P5 juga menghentikan Feed dan memeriksa pembacaan hazard serta kembalinya endpoint SSE. Tes itu belum mengassert kenaikan dan pengurasan backlog secara numerik. Bukti backlog pada laporan berasal dari gambar manual Notification di atas, bukan dari perluasan arti status PASS pada ringkasan otomatis.

6.4.3 Consumer Tambahan pada Exchange yang Sama

Pada sesi pengujian lainnya, Webhook Provider dijalankan ketika producer dan layanan lain sudah tersedia. Log `P5/4-1.png` memperlihatkan pembuatan container/volume webhook, inisialisasi SQLite, serta koneksi consumer ke queue `bnpb.webhook-provider.v1`. Tangkapan layar broker kemudian memperlihatkan queue webhook bersama Feed dan Notification; ketiga queue utama mempunyai laju incoming, delivery, dan ack.

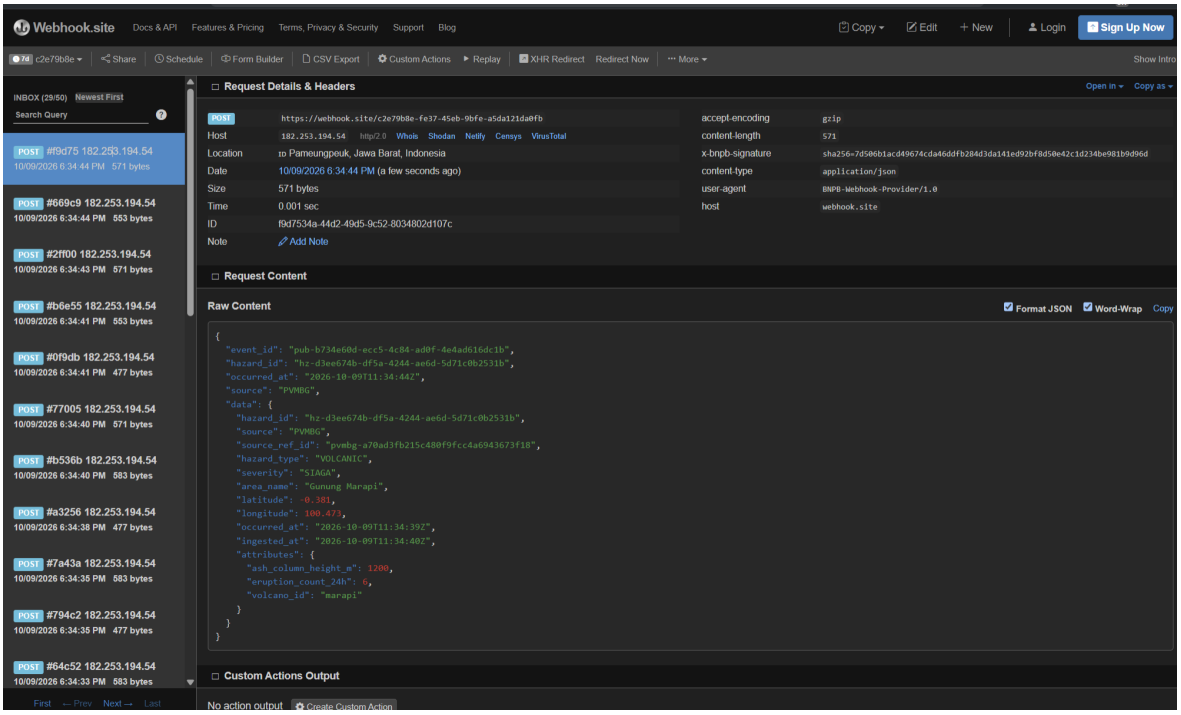


Gambar 6.4: M5: tiga queue consumer aktif setelah Webhook Provider tersedia.

Callback HTTP yang diterima memuat `event_id`, `hazard_id`, sumber PVMBG, dan snapshot hazard. Hal ini menunjukkan bahwa jalur consumer webhook menghasilkan callback data, bukan sekadar queue yang berhasil dibuat. Endpoint callback eksternal pada bukti hanya digunakan untuk demo.

Bukti alternatif merekam penambahan queue baru dan binding ke exchange fanout melalui Management API. Screenshot binding sendiri belum menunjukkan pengambilan pesan oleh consumer. Tes otomatis `test_04_dynamic_new_consumer_addition_zero_touch` melengkapi bagian ini dengan membuat queue unik, mengikatnya ke exchange, menunggu serta mengambil satu pesan, lalu menghapus queue uji. Operasi itu tidak mengubah kode producer. Bukti yang tersedia tidak menyimpan perbandingan ID/start time container Aggregator pada seluruh sesi, sehingga tidak digunakan untuk menyimpulkan riwayat restart producer di luar interval observasi.

Pengujian DLQ, injeksi publication duplikat, pemulihan setelah broker dibuat ulang, dan keberhasilan push pada perangkat tidak direkam dalam rangkaian M5 ini. Batasan at-least-once, retensi replay Feed, volume broker, dan efek samping FCM yang dijelaskan sebelumnya tetap berlaku. Artefak E4 adalah ringkasan hasil tes; rincian klaim harus dibaca bersama assertion pada `tests/test_problem_5.py` dan gambar manual, sebagaimana inventaris pada `docs/tex/evidence/manifest.json`.



Gambar 6.5: M5: payload hazard diterima pada endpoint callback Webhook Provider.

Tabel 6.2: Ringkasan bukti P5

Kriteria	**Hasil yang didukung artefak**
Publication melalui broker	Trace publish–consume dan exchange fanout diperlihatkan.
Dua consumer independen	Feed dan Notification menerima melalui queue masing-masing dan terhubung pada trace publication.
Consumer offline/pulih	Notification nol consumer dengan backlog 23; sesudah aktif, backlog yang sempat melebihi 100 turun menjadi nol; Feed tetap aktif.
Consumer tambahan	Webhook memiliki queue dan menerima callback; tes queue dinamis juga mengambil pesan tanpa perubahan kode producer.

Daftar Pustaka

[1] Tim Pengampu IF4031. *IF4031 – Tugas Besar – M1*. Dokumen spesifikasi tugas yang diberikan, bagian P1–P5 dan ketentuan laporan.

[2] Kelompok nasigoreng. *Tugas-Besar-AAT-M1*, snapshot repository commit `4a1973b`, 10 Oktober 2026. Kontrak pada `contracts/`, implementasi pada `services/`, orkestrasi pada `compose.yaml`, serta artefak pengujian pada `docs/tex/evidence/`.

[3] OpenID Foundation. *OpenID Connect Core 1.0 incorporating errata set 2*. `https://openid.net/specs/openid-connect-core-1_0.html`.

[4] N. Sakimura, J. Bradley, dan N. Agarwal. *RFC 7636: Proof Key for Code Exchange by OAuth Public Clients*. 2015. `https://www.rfc-editor.org/info/rfc7636/`.

[5] B. Campbell dkk. *RFC 8693: OAuth 2.0 Token Exchange*. 2020. `https://www.rfc-editor.org/info/rfc8693/`.

[6] PostgreSQL Global Development Group. *PostgreSQL Documentation: JSON Types*. `https://www.postgresql.org/docs/current/datatype-json.html`. Referensi konsep JSONB; container implementasi menggunakan PostgreSQL 17.

[7] RabbitMQ. *Consumer Acknowledgements and Publisher Confirms*. `https://www.rabbitmq.com/docs/confirms`.